

Federated Distributed Key Generation

Stanislaw Baranski  Julian Szymanski 

Abstract—Distributed Key Generation (DKG) underpins threshold cryptography across many domains: decentralized custody and wallets, validator/committee key ceremonies, cross-chain bridges, threshold signatures, secure multiparty computation, and internet voting.

Classical (t, n) -DKG assumes a fixed set of n parties and a global threshold t , and effectively requires full, timely participation; when actual participation or availability deviates, setups abort or must be rerun—a severe problem in open or ad-hoc, time-critical settings, especially when n is large and node availability is unpredictable.”

We introduce *Federated Distributed Key Generation (FDKG)*, inspired by Federated Byzantine Agreement, that makes participation optional and trust *heterogeneous*. Each participant selects a personal guardian set $\mathcal{G}_i$ of size k and a local threshold t ; its partial secret can later be reconstructed either by itself or by any t of its guardians. FDKG generalizes PVSS-based DKG and completes both *generation* and *reconstruction* in a single broadcast round each, with total communication $\Theta(nk)$ and (in the worst case) $\mathcal{O}(n^2)$ for reconstruction.

Our security analysis shows that: (i) *Generation* achieves correctness, privacy, and robustness under standard PVSS-based DKG assumptions; and (ii) *Reconstruction* provides liveness and privacy characterized by the *guardian-set topology* $\{\mathcal{G}_i\}$. Liveness holds provided that, for every participant i , the adversary does not control i together with at least $k - t + 1$ of its guardians. Conversely, reconstruction privacy holds unless the corrupted set itself is reconstruction-capable.

In a scenario with $n=100$ potential parties ($|\mathbb{D}|=50$, $k=40$, 80% retention for tallying), FDKG distribution broadcasts ≈ 336 kB and reconstruction ≈ 525 kB. Client-side Groth16 proving per participant is ≈ 5 s for distribution, and 0.6–29.6 s for reconstruction depending on the number of revealed shares.

I. INTRODUCTION

Distributed Key Generation (DKG) is a multi-party protocol that lets a group of mutually distrustful parties jointly create a public/secret key pair and hold it via threshold shares: any t out of n parties can reconstruct or use the secret, while fewer learn nothing [1, 2]. By shifting key creation from a single entity to a committee, DKG removes single point of failure for core operations such as decryption or signing. DKGs are widely used across threshold encryption and signatures [1–5], consensus protocols [6], multi party computation (MPC) [7–9], distributed randomness beacons (DRB) [10–12], cryptocurrency wallets [13, 14] and internet voting protocols [15–18].

However, a standard (t, n) -DKG protocol [1, 2] requires n to be known a priori to determine polynomial degrees and sharing parameters. In settings where participation is voluntary (e.g., public blockchains or ad hoc networks), the realized participant count n' may only become clear after an

initial interaction phase. If n' deviates from the anticipated n , the pre-selected global threshold t may be unattainable ($t > n'$) or misaligned with the actual corruption/availability assumptions; in either case the system must be reparameterized and restarted, incurring coordination cost and delay that are often unacceptable in time-critical deployments. Furthermore, **traditional DKGs enforce uniform (homogeneous) trust among all n parties, discarding the organic trust structure that could otherwise be leveraged to improve security and availability.**

To overcome these limitations, we introduce *Federated Distributed Key Generation (FDKG)*, a protocol that draws conceptual inspiration from Federated Byzantine Agreement (FBA) protocols [19], designed for scenarios with uncertain participation and heterogeneous trust relationships. FDKG allows any subset of potential parties to participate in the key generation process. Its core innovation lies in *Guardian Sets*: each participating party P_i independently selects a trusted subset of k other parties (its guardians) and defines a local threshold t necessary for these guardians to reconstruct P_i 's partial secret key.

Formally, an FDKG protocol involves n potential parties. A subset $\mathbb{D} \subseteq \mathbb{P}$ chooses to participate in generating partial secret keys. Each participant $P_i \in \mathbb{D}$ distributes its partial secret key d_i among its chosen guardian set $\mathcal{G}_i$ of size k using a local (t, k) -threshold secret sharing scheme. The global secret key $\mathbf{d}$ is the sum of all valid partial secrets d_i . During reconstruction, $\mathbf{d}$ can be recovered if, for each $P_i \in \mathbb{D}$, either P_i itself provides d_i , or at least t of its guardians in $\mathcal{G}_i$ provide their respective shares. FDKG generalizes traditional (t, n) -DKG, which can be viewed as a special case where all n parties participate and each party's guardian set comprises all other $n - 1$ parties.

This novel approach offers several advantages:

- **Optional Participation:** FDKG accommodates any subset of parties $P_i \in \mathbb{D}$ joining the key generation process without a prior commitment phase, improving UX by reducing interaction and avoiding costly reparameterizations/restarts when $n' \neq n$.
- **Heterogeneous Trust Model:** By allowing each participant to define its own guardian set $\mathcal{G}_i$, FDKG supports diverse and evolving trust relationships, making it suitable for decentralized settings where trust is not uniform. This is illustrated in Figure 1, which contrasts FDKG's federated trust with centralized and traditional distributed trust models.
- **Resilience to Node Unavailability:** The use of local thresholding via guardian sets ensures that a participant's contribution to the global key can be recovered even if the participant itself becomes unavailable, provided a sufficient number of its designated guardians remain honest and active.

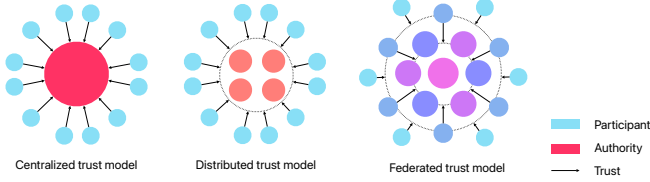


Figure 1: Comparison of trust models in distributed systems: centralized trust (one authority), distributed trust (multiple authorities), and federated trust (peer-to-peer with individually chosen trusted groups).

In practice, FDKG succeeds in settings where classical (t, n) -DKG cannot tolerate participation uncertainty without costly restarts and timeline slips. Consider a public voting with a 24-hour enrollment window for talliers. The organizer publishes a list of $n = 60$ eligible talliers and plans a classical DKG with $t = 40$. After the window, only $n' = 37$ complete enrollment and verification. Since classical DKG pre-fixes t against the planned n , **the chosen $t = 40$ is unattainable ($t > n'$), so setup must abort or be restarted with new parameters.** In FDKG, the same upper bound $n = 60$ is published, but Round 1 accepts any subset $\mathbb{D}$ of those who post valid transcripts (here $|\mathbb{D}| = 37$).

A time-critical disclosure scenario similarly benefits. A whistleblower wants to share files with up to $n = 12$ independent journalists; the case is urgent, so he sets a 6-hour enrollment window. His goal is to include as many as possible by the deadline, encrypt once the window closes, and avoid being blocked by a few non-responders. With FDKG, the joint key emerges from the actual participants $\mathbb{D}$, so progress does not hinge on every individual being present. Classical DKG, by contrast, requires full participation; if fewer than the planned n attend within 6 hours, the setup fails and must be rerun—missing the deadline the whistleblower cannot extend.

In this paper, we provide a detailed specification of the FDKG protocol, a formal security analysis within the simulation-based paradigm (Section V), and demonstrate its practical utility through an application to an internet voting scheme. Our primary contribution is the FDKG framework itself, which enables robust and verifiable key generation with adaptable trust for dynamic networks.

This paper makes the following contributions:

- We formally define the FDKG protocol.
- We provide a security analysis in the simulation-based framework, establishing Correctness, Privacy, and Robustness for key generation, and Liveness for reconstruction (Section V).
- We demonstrate FDKG’s performance and resilience through extensive simulations under various network conditions, participation rates, and retention scenarios (Section VI), and evaluate the computational and communication costs of its cryptographic components (Section VIII).
- We integrate FDKG into an end-to-end internet voting scheme, showcasing its practical application using threshold ElGamal encryption and NIZK-based verifiability

(Section VII). A corresponding open-source implementation, together with reproducibility scripts, is provided.

The remainder of this paper is organized as follows. Section II reviews related work in DKG. Section III presents the cryptographic primitives. Section IV describes the FDKG protocol. Section V provides the security analysis. Section VI discusses simulation results. Section VII details the voting application. Section VIII assesses performance. Section IX outlines deployment strategies. Finally, Section X discusses limitations and future work, and Section XI offers concluding remarks.

II. RELATED WORK

Foundational DKG protocols, such as [1, 2], established the core paradigm: n participants, each acting as a dealer for a partial secret, combine their contributions to form a single joint public key and distributed secret shares. These protocols are typically based on Verifiable Secret Sharing (VSS) [30], which allows participants to verify the integrity of their received shares.

Subsequent research has extensively explored DKG, focusing on various aspects such as robustness against adversarial behavior, efficiency, round complexity, security, and applicability in different network models (synchronous vs. asynchronous) [20]. Considerable effort has been dedicated to enhancing DKG for reliable operation in challenging environments like the internet, with a focus on asynchronous settings and high thresholds of corruptions [31–34]. A significant body of work focuses on Publicly Verifiable Secret Sharing (PVSS) [35, 36], where the correctness of shared information can be verified by any observer, not just the share recipients. PVSS is crucial for non-interactive DKG protocols, where parties broadcast their contributions along with proofs, minimizing interaction rounds. Groth’s work on DKG and key resharing [24] presents efficient constructions leveraging pairing-based cryptography and NIZK proofs, aiming for minimal rounds and public verifiability, which is a common goal for modern DKG deployed on transparent ledgers or bulletin boards.

The recent SoK by Bacho and Kavousi [20] identifies two primary axes for classifying DKG protocols: the underlying network model and the secret sharing mechanism employed. To accurately position FDKG, we adopt this framework and establish a coherent baseline for comparison.

First, DKG protocols are distinguished by their network model assumption. **Synchronous protocols** assume bounded message delays, allowing them to operate in well-defined rounds. In contrast, **asynchronous protocols** [37] are designed to tolerate unbounded message delays, a significantly stronger guarantee that requires more complex consensus mechanisms and often results in higher communication overhead. As FDKG is designed for synchronous environments, we omit asynchronous DKG protocols from our direct comparison to ensure an evaluation of protocols operating under similar network guarantees.

A second critical distinction is the protocol’s resilience to key biasing by a rushing adversary. Modern research differentiates between **biasable** protocols and **unbiasable** (or

Table I: Comparison of synchronous, biasable DKG protocols. Core columns follow Bacho–Kavousi [20]. We split “Corrupt” into *Privacy* and *Liveness*, and add *Trust* and *Setup*. Privacy: adversary cannot learn the shared secret. Liveness: adversary cannot prevent honest parties from recovering the shared secret. Let $\mathcal{C} \subseteq \mathbb{P}$ be the statically corrupted set, and let R be the family of *reconstruction-capable sets* (as in Eq. (1)). **Communication** is total broadcast cost; $BC_n(b)$ is the cost for one party to broadcast b bits to n parties; k denotes the guardian set size used in the FDKG. **Rounds** denotes the number of broadcast rounds. **Sharing** indicates the secret-sharing type (VSS, PVSS, or APVSS). **Trust Model**: “Homo” (homogeneous) means a global, uniform trust assumption; “Hetero” (heterogeneous) allows participant-defined, non-uniform trust. **Setup**: “Required” means a fixed participant set must take part; “Optional” discovers participants from a larger pool.

Protocol	Year	Privacy	Liveness	Communication	Rounds	Sharing	Trust	Setup
Pedersen [1]	1991	$ \mathcal{C} < t < n/2$	$ \mathcal{C} < n - t$	$n BC_n(\lambda n)$	$3 \cdot BC_n$	VSS	Homo	Required
Fouque–Stern [21]	2001	$ \mathcal{C} < t < n$	$ \mathcal{C} < n - t$	$n BC_n(\lambda n)$	$1 \cdot BC_n$	PVSS	Homo	Required
Kate et al. [22]	2010	$ \mathcal{C} < t < n/2$	$ \mathcal{C} < n - t$	$n BC_n(\lambda + \delta \lambda n)$	$3 \cdot BC_n$	VSS	Homo	Required
Gürkan et al. [23]	2021	$ \mathcal{C} < t < \log n$	$ \mathcal{C} < n - t$	$n BC_n(\lambda) + \ell^2 BC_n(\lambda n)$	$\ell \cdot BC_n$	APVSS	Homo	Required
Groth [24]	2021	$ \mathcal{C} < t < n$	$ \mathcal{C} < n - t$	$n BC_n(\lambda^2 n)$	$1 \cdot BC_n$	PVSS	Homo	Required
Kate et al. [25]	2023	$ \mathcal{C} < t < n$	$ \mathcal{C} < n - t$	$n BC_n(\lambda n)$	$1 \cdot BC_n$	PVSS	Homo	Required
Cascudo–David [26]	2023	$ \mathcal{C} < t < n$	$ \mathcal{C} < n - t$	$n BC_n(\lambda n)$	$1 \cdot BC_n$	PVSS	Homo	Required
Feng et al. [27]	2023	$ \mathcal{C} < t < n/2$	$ \mathcal{C} < n - t$	$\kappa BC_n(\lambda n) + n \lambda \kappa$	$2 \cdot BC_n$	VSS	Homo	Required
Bacho et al. [28]	2023	$ \mathcal{C} < t < n/2$	$ \mathcal{C} < n - t$	$BC_n(\lambda n) + n^2 \lambda \log n$	$4 \cdot BC_n$	APVSS	Homo	Required
Feng et al. [29]	2024	$ \mathcal{C} < t < n/2$	$ \mathcal{C} < n - t$	$\sqrt{n} BC_n(\lambda n \kappa)$	$2 \cdot BC_n$	PVSS	Homo	Required
FDKG (this work)	2024	$\mathcal{C} \not\subseteq R$	$\exists S \in R : S \subseteq \mathbb{P} \setminus \mathcal{C}$	$n BC_n(\lambda k)$	$1 \cdot BC_n$	PVSS	Hetero	Optional

”fully secure”) protocols [38]. Fully secure DKGs guarantee that the final public key’s distribution is uniform and cannot be influenced by a malicious minority. This robustness typically requires additional rounds of communication (e.g., commit-reveal schemes). FDKG, in line with many practical protocols, prioritizes simplicity and a minimal round count, and is therefore biasable. Consequently, to evaluate FDKG against schemes with similar design trade-offs, we focus our comparison on the class of biasable protocols.

Furthermore, it is important to distinguish DKG from Dynamic and Proactive Secret Sharing (DPSS) schemes [39]. DPSS protocols are designed for the long-term lifecycle management of a secret, enabling committees to evolve and shares to be proactively refreshed to defend against adaptive adversaries. Since FDKG focuses on the initial *bootstrapping* of a key, DPSS protocols address a different problem domain and are thus not directly included in our setup-focused analysis.

Our design draws conceptual inspiration from Federated Byzantine Agreement (FBA) protocols—such as the Stellar Consensus Protocol [19]—where nodes declare local “quorum slices” (trusted peers), enabling open participation without a globally pre-defined validator set. FDKG borrows this form of heterogeneous, participant-defined trust: each party i specifies a guardian set G_i with local threshold t , and a party included in many guardian sets becomes operationally more critical for liveness. The semantics, however, differ fundamentally. In FBA, safety and liveness are defined via quorum intersection and concern *consensus* over a ledger; in FDKG, properties are defined via the reconstruction family R (defined in Section V) and concern a one-shot cryptographic reconstruction predicate. Thus, while both systems use local trust slices, FDKG makes no consensus claims; our guarantees (Theorems 3–4) address reconstruction privacy and liveness rather than agreement.

To establish a coherent baseline and facilitate a sound comparison, we therefore narrow our focus to the class of

synchronous, biasable DKG protocols. Within this class, different secret sharing mechanisms are used, as detailed in the [20]. These include complaint-based VSS, non-interactive Publicly Verifiable Secret Sharing (PVSS) [21, 40] (enabling one-round designs), and Aggregatable PVSS (APVSS) [23, 41] (allowing transcript aggregation). Beyond the sharing mechanism, protocols differ in three comparison-relevant dimensions that we report in Table I: (i) the *privacy security* and *liveness tolerance* for corrupted parties; (ii) the round complexity, ranging from *one* broadcast round (PVSS) up to *four* rounds (multi-phase APVSS/complaint flows); and (iii) the *communication overhead*.

Our comparative analysis, presented in Table I, evaluates FDKG against this established baseline. The table reveals a consistent architectural paradigm across all surveyed protocols. To the best of our knowledge, FDKG is the first protocol in its class to break from this paradigm in two fundamental ways. First, it introduces a **heterogeneous trust model**, moving away from the assumption that trust is global and uniform (“Homo”). Second, it supports **optional participation**, whereas all prior schemes require the participation of a fixed, pre-defined set of members (“Required”). These architectural changes are what enable FDKG to provide key generation in time-critical and ad-hoc environments.

III. PRELIMINARIES

This section introduces the cryptographic primitives, concepts, and notations foundational to our Federated Distributed Key Generation (FDKG) protocol.

A. Notation

Let λ denote the computational security parameter. We work within a cyclic group $\mathbb{G}$ of prime order q generated by $G \in \mathbb{G}$. Group operations are written multiplicatively, and

scalar multiplication is denoted by G^a for $a \in \mathbb{Z}_q$. We assume the Discrete Logarithm Problem (DLP) is hard in $\mathbb{G}$. All algorithms are assumed to be Probabilistic Polynomial-Time (PPT) unless stated otherwise. Sampling a random element x from a set S is denoted by $x \leftarrow S$. We denote the set of potential parties as $\mathbb{P} = \{P_1, \dots, P_n\}$.

B. Shamir's Secret Sharing (SSS)

A (t, n) -threshold Shamir Secret Sharing scheme over the finite field $\mathbb{Z}_q$ (where q is prime and $q > n$) allows a dealer to share a secret $s \in \mathbb{Z}_q$ among n parties $P_1, \dots, P_n$ such that any subset of t or more parties can reconstruct s , while any subset of fewer than t parties learns no information about s .

- $(s_1, \dots, s_n) \leftarrow \text{Share}(s, t, n)$: The dealer chooses a random polynomial $f(X) = \sum_{k=0}^{t-1} a_k X^k \in \mathbb{Z}_q[X]$ of degree $t-1$ such that $f(0) = s$. For each party P_i ($i \in [n]$), the dealer computes the share $s_i = f(i)$.
- $s \leftarrow \text{Reconstruct}(I, \{s_i\}_{i \in I})$: Given a set $I \subseteq [n]$ of at least t indices and their corresponding shares $\{s_i\}_{i \in I}$, this algorithm computes $s = \sum_{i \in I} \lambda_i s_i$, where λ_i are the Lagrange coefficients for the set I evaluated at 0.

SSS provides perfect privacy against adversaries controlling fewer than t parties.

C. Verifiable and Publicly Verifiable Secret Sharing (VSS/PVSS)

VSS schemes enhance SSS by allowing parties to verify the consistency of their shares relative to some public information. PVSS strengthens this by enabling *any* entity to perform this verification using only public data [35, 36]. PVSS schemes typically involve the dealer encrypting each share s_i under the recipient P_i 's public key pk_i and publishing these ciphertexts $C_i = \text{Enc}(\text{pk}_i, s_i)$ along with public commitments to the secret polynomial (e.g., $\{A_k = G^{a_k}\}$) and a public proof π of consistency. Our FDKG protocol employs a PVSS scheme where each participant acts as a dealer.

DKG from PVSS: In a typical PVSS-based DKG [38], each party P_k acts as a dealer for a partial secret $d_k = f_k(0)$. P_k uses PVSS to distribute shares $d_{k,j} = f_k(j)$ to all parties P_j , publishing commitments $\{E_{k,l} = G^{a_{k,l}}\}$, encrypted shares $\{C_{k,j}\}$, and a proof π_k . The final secret key is $d = \sum_k d_k$, the public key is $E = \prod_k E_{k,0}$, and party P_i 's share is $d_i = \sum_k d_{k,i}$.

D. Public Key Encryption (PKE)

A public-key encryption scheme $\mathcal{E} = (\text{KeyGen}, \text{Enc}, \text{Dec})$ consists of three algorithms:

- $(\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(1^\lambda)$: Generates a public key pk_i and a secret key sk_i .
- $c \leftarrow \text{Enc}(\text{pk}_i, m, \text{rand})$: Encrypts message m under pk_i using provided randomness rand to produce ciphertext c .
- $m / \perp \leftarrow \text{Dec}(\text{sk}_i, c)$: Decrypts ciphertext c using secret key sk_i to recover message m or outputs failure $\perp$.

We require **IND-CPA** security (Indistinguishable under Chosen Plaintext Attack).

E. Non-Interactive Zero-Knowledge (NIZK) Proofs

NIZK proofs allow proving the truth of an NP statement $x \in \mathcal{L}$ (for relation $\mathcal{R}$) without interaction and without revealing the witness w .

NIZKs require a setup phase. Different models exist regarding the nature and trust assumptions of this setup:

- **Structured Reference String (SRS) / Trusted Setup:** Systems like Groth16 [42] require a CRS generated with specific mathematical structure. This structure is essential for proof succinctness and efficiency but typically requires a trusted setup ceremony (or an MPC equivalent) to generate the CRS without revealing a trapdoor. The CRS is specific to a particular circuit/relation.
- **Universal / Updatable Setup:** Systems like PLONK [43] or Marlin [44] use a CRS that can be used for any circuit up to a certain size (universal) and may allow the CRS to be updated securely without repeating a full trusted ceremony (updatable). They still require an initial setup phase, often trusted.
- **Transparent Setup / Random Oracle Model (ROM):** Systems like STARKs [45] require minimal setup assumptions, often relying only on public randomness (transparent) or operating in the idealized Random Oracle Model (ROM), where a public hash function is modeled as a truly random function. These avoid complex or trusted setup ceremonies.

Our current implementation utilizes Groth16 due to its efficiency and widespread tooling, thus operating in the CRS model and requiring a trusted setup. However, the FDKG framework itself is compatible with other NIZK systems.

A NIZK system (e.g., in the CRS model) provides:

- $\text{crs} \leftarrow \text{Setup}(1^\lambda, \mathcal{R})$: Generates the public CRS.
- $\pi \leftarrow \text{Prove}(\text{crs}, x, w)$: Produces proof π .
- $b \leftarrow \text{Verify}(\text{crs}, x, \pi)$: Outputs 1 (accept) or 0 (reject).

We require standard Completeness, computational Soundness, and computational Zero-Knowledge.

F. Encrypted Channels for Share Distribution

Protocols like PVSS and DKG often require confidential transmission of secret shares (s_i) to specific recipients (P_i) over a public broadcast channel. This is achieved using an IND-CPA secure PKE scheme. Each potential recipient P_i has a key pair $(\text{pk}_i, \text{sk}_i)$ where pk_i is public. To send s_i to P_i , the sender computes $c = \text{Enc}(\text{pk}_i, s_i, \text{rand})$ and broadcasts c . Only P_i can compute $s_i = \text{Dec}(\text{sk}_i, c)$.

For circuit efficiency with Groth16, we use an ElGamal variant over BabyJub [46–48]. The specific ElGamal variant encryption and decryption algorithms used in our implementation are detailed in Algorithm1 and Algorithm2, respectively.

G. Summary of Key Notations

Our implementation instantiates IND-CPA PKE with an ElGamal variant over BabyJub (Edwards curve) and NIZKs with Groth16. The DLP hardness assumption is therefore the discrete-log assumption in the BabyJub subgroup; IND-CPA security reduces to the DDH assumption in that group.

Algorithm 1: ElGamal Encryption $\text{Enc}(\text{pk}_i, m, [k, r])$

-
- 1 **Input:** Recipient public key pk_i , scalar message $m \in \mathbb{Z}_q$, randomness $(k, r) \in \mathbb{Z}_q \times \mathbb{Z}_q$
 - 2 **Output:** Ciphertext tuple (C_1, C_2, Δ)
 - 3 $C_1 = G^k$
 - 4 $M = G^r$
 - 5 $C_2 = (\text{pk}_i)^k \cdot M$
 - 6 $\Delta = M \cdot x - m \pmod{q}$
 - 7 **return** (C_1, C_2, Δ)
-

Algorithm 2: ElGamal Decryption $\text{Dec}(\text{sk}_i, (C_1, C_2, \Delta))$

-
- 1 **Input:** Recipient secret key sk_i , ciphertext (C_1, C_2, Δ)
 - 2 **Output:** Scalar message $m \in \mathbb{Z}_q$
 - 3 $M = C_2 \cdot (C_1^{\text{sk}_i})^{-1}$
 - 4 $m = M \cdot x - \Delta \pmod{q}$
 - 5 **return** m
-

Table II: Key Notations Used in the FDKG Protocol

Notation	Description
n	Total number of potential parties in the system $\mathbb{P}$.
$\mathbb{P}$	Set of all potential parties $\{P_1, \dots, P_n\}$.
P_i	Identifier for party i .
$\mathbb{G}, G, q$	Cyclic group, its generator, and prime order.
pk_i, sk_i	PKE public/secret key pair for P_i .
crs	Common Reference String for NIZK proofs.
π	Generic symbol for a NIZK proof (e.g., π_{FDKG_i}).
$\mathcal{G}_i$	Guardian set chosen by participant P_i .
t	Threshold for share reconstruction within a guardian set.
k	Size of each participant's chosen guardian set.
$\mathbb{D}$	Set of valid parties in the generation phase.
d_i, E_i	Partial secret/public key of participant $P_i \in \mathbb{D}$. ($E_i = G^{d_i}$).
$\mathbf{d}, \mathbf{E}$	Global secret/public key aggregated from partial keys. ($\mathbf{d} = \sum d_i, \mathbf{E} = \prod E_i = G^{\mathbf{d}}$).
$s_{i,j}$	SSS share of d_i for guardian $P_j \in \mathcal{G}_i$.
$C_{i,j}$	Ciphertext of $s_{i,j}$ encrypted under pk_j .
$\text{Enc}(\text{pk}, \cdot, \cdot)$	PKE encryption algorithm.
$\text{Dec}(\text{sk}, \cdot)$	PKE decryption algorithm.
$\mathbb{V}$	Set of parties participating in the voting phase.
B_i, v_i, r_i	Voter P_i 's encrypted ballot, encoded vote, and blinding factor.
$\mathbb{T}$	Set of parties participating in the online tally phase.
$C1, C2$	Aggregated ElGamal components of all cast ballots.
PD_i	Partial decryption of $C1$ by P_i using d_i .
$[\text{PD}_j]_i$	Partial decryption of $C1$ by guardian P_i using share $s_{j,i}$.

Groth16 soundness/zero-knowledge rely on pairing-friendly curve security (BN254 in our implementation) and a trusted SRS for the given circuits. The security reductions in Section V use only these abstract properties; other IND-CPA PKEs or NIZKs (PLONK/Marlin/STARKs) could be substituted with corresponding trade-offs in proof size and proving time.

IV. FEDERATED DISTRIBUTED KEY GENERATION (FDKG)

Federated Distributed Key Generation (FDKG) generalizes DKG to dynamic participation: any subset of parties can generate a joint key and delegate reconstruction to self-chosen guardian sets.

A. Protocol Setting and Goal

We consider a set $\mathbb{P} = \{P_1, \dots, P_n\}$ of n potential parties. The system operates over a cyclic group $\mathbb{G}$ of prime order q with generator G . We assume a Public Key Infrastructure (PKI) where each party P_i possesses a long-term secret key sk_i and a corresponding public key pk_i for an IND-CPA secure public-key encryption scheme (Enc, Dec) (Section III-D), used for secure share distribution. Communication occurs via a broadcast channel. The protocol also utilizes a Non-Interactive Zero-Knowledge (NIZK) proof system $(\text{Setup}, \text{Prove}, \text{Verify})$ (Section III-E) which is assumed to be complete, computationally sound, and zero-knowledge, operating with a common reference string crs .

The goal of FDKG is for any subset of participating parties $\mathbb{D} \subseteq \mathbb{P}$ to jointly generate a global public key $\mathbf{E} \in \mathbb{G}$ such that the corresponding secret key $\mathbf{d} \in \mathbb{Z}_q$ (where $\mathbf{E} = G^{\mathbf{d}}$) is shared among them. Specifically, each participant $P_i \in \mathbb{D}$ generates a partial key pair (d_i, E_i) where $E_i = G^{d_i}$. The global key pair is formed by aggregation: $\mathbf{d} = \sum_{i \in \mathbb{D}} d_i$ and $\mathbf{E} = \prod_{i \in \mathbb{D}} E_i$. Each participant P_i uses a local (t, k) -Shamir Secret Sharing (SSS) scheme (Section III-B) to distribute its partial secret d_i among a self-selected guardian set $\mathcal{G}_i \subset \mathbb{P} \setminus \{P_i\}$ of size k .

B. The FDKG Protocol (Π_{FDKG})

The protocol proceeds in two rounds: Generation and Reconstruction.

1) *Round 1: Generation phase:* Any party $P_i \in \mathbb{P}$ may (optionally) participate:

- 1) **Guardian Selection:** Choose $\mathcal{G}_i \subset \mathbb{P} \setminus \{P_i\}$ with $|\mathcal{G}_i| = k$.
- 2) **Partial Key Generation:** Sample partial secret $d_i \leftarrow \mathbb{Z}_q$. Compute $E_i = G^{d_i}$.
- 3) **Share Generation:** Generate shares $(s_{i,1}, \dots, s_{i,n}) \leftarrow \text{Share}(d_i, t, n)$.
- 4) **Share Encryption:** Initialize empty lists for ciphertexts $\mathbb{C}_i$ and randomness $\mathbb{R}_i$. For each $P_j \in \mathcal{G}_i$:
 - Sample PKE randomness: $(k_{i,j}, r_{i,j}) \leftarrow \mathbb{Z}_q \times \mathbb{Z}_q$.
 - Encrypt: $C_{i,j} \leftarrow \text{Enc}_{\text{pk}_j}(s_{i,j}, [k_{i,j}, r_{i,j}])$.
 - Store: Add $C_{i,j}$ to $\mathbb{C}_i$ and add $(k_{i,j}, r_{i,j})$ to $\mathbb{R}_i$.
- 5) **Proof Generation:** Let the public statement be $x_i = (E_i, \{\text{pk}_j\}_{P_j \in \mathcal{G}_i}, \mathbb{C}_i)$. Let the witness be $w_i = (d_i, \{s_{i,j}\}_{P_j \in \mathcal{G}_i}, \mathbb{R}_i)$. Generate $\pi_{\text{FDKG}_i} \leftarrow \text{Prove}(\text{crs}, x_i, w_i)$ (see Appendix C).
- 6) **Broadcast:** Broadcast $(P_i, E_i, \mathcal{G}_i, \mathbb{C}_i, \pi_{\text{FDKG}_i})$.

Post-Round 1 Processing (by any observer):

- Initialize $\mathbb{D} = \emptyset$.
- For each received tuple $(P_i, E_i, \mathcal{G}_i, \mathbb{C}_i, \pi_i)$:
 - Construct statement $x_i = (E_i, \{\text{pk}_j\}_{P_j \in \mathcal{G}_i}, \mathbb{C}_i)$.
 - Verify proof $b \leftarrow \text{Verify}(\text{crs}, x_i, \pi_i)$.
 - If $b = 1$, add P_i to $\mathbb{D}$ and store $(E_i, \mathcal{G}_i, \mathbb{C}_i) \leftarrow (E_i, \mathcal{G}_i, \mathbb{C}_i)$.
- Compute $\mathbf{E} = \prod_{P_i \in \mathbb{D}} E_i$.
- Public state: $\mathbb{D}, \mathbf{E}, \{E_i, \mathcal{G}_i, \mathbb{C}_i\}_{P_i \in \mathbb{D}}$.

2) **Round 2: Reconstruction Phase: Online Reconstruction (by parties $P_i \in \mathbb{T} \subseteq \mathbb{P}$):**

- 1) **Reveal Partial Secret:** If $P_i \in \mathbb{D} \cap \mathbb{T}$:
 - Generate $\pi_{PS_i} \leftarrow \text{Prove}(\text{crs}, E_i, d_i)$ (see Appendix D).
 - Broadcast $(P_i, \text{secret}, d_i, \pi_{PS_i})$.
- 2) **Reveal Received Shares:** For each $C_{j,i}$ (where $P_j \in \mathbb{D}, P_i \in \mathcal{G}_j$):
 - Decrypt share: $s_{j,i} \leftarrow \text{Dec}(\text{sk}_i, C_{j,i})$.
 - Generate proof: $\pi_{SPS_{j,i}} \leftarrow \text{Prove}(\text{crs}, (C_{j,i}, \text{pk}_i, s_{j,i}), \text{sk}_i)$ (see Appendix E).
 - Broadcast $(P_i, \text{share}, P_j, s_{j,i}, \pi_{SPS_{j,i}})$.

Offline Reconstruction (by any observer):

- 1) Initialize empty list $D = \emptyset$.
- 2) For each participant $P_i \in \mathbb{D}$:
 - **Direct Secrets Reconstruction:** Look for $(P_i, \text{secret}, d_i, \pi_{PS_i})$. If $1 = \text{Verify}(\text{crs}, E_i, \pi_{PS_i})$, add d_i to D ;
 - **Share-Based Reconstruction**
 - a) Initialize empty list S_i . For each $(P_k, \text{share}, P_i, s_{i,j}, \pi_{SPS_{i,j}})$ where $P_k = P_j$ and $C_{i,j}$ exists: $\text{Verify } b \leftarrow \text{Verify}(\text{crs}, (C_{i,j}, \text{pk}_j, s_{i,j}), \pi_{SPS_{i,j}})$. If $b = 1$, add $(P_j, s_{i,j})$ to S_i .
 - b) Let $I_i = \{P_j \mid (P_j, \cdot) \in S_i\}$. If $|I_i| \geq t$: Select $I'_i \subseteq I_i, |I'_i| = t$. Compute $d_i \leftarrow \text{Reconstruct}(I'_i, \{s_{i,j} \mid (P_j, s_{i,j}) \in S_i \text{ and } P_j \in I'_i\})$. If $G^{d_i} = E_i$, add d_i to D .
- 3) **Compute Global Key:** If D contains a valid entry for every $P_i \in \mathbb{D}$, compute $\mathbf{d} = \sum_{d_i \in D} d_i$. Output $\mathbf{d}$. Else output failure.

C. Communication Complexity

The communication complexity depends on the number of participants $|\mathbb{D}|$ and the size of guardian sets k .

- **Round 1 (Generation):** Each participant $P_i \in \mathbb{D}$ broadcasts its partial public key E_i (1 group element), k encrypted shares $\{C_{i,j}\}$ (size depends on PKE scheme, in our scheme $k \times (2|\mathbb{G}| + |\mathbb{Z}_q|)$), and one NIZK proof π_{FDKG_i} . Assuming $|\mathbb{D}| \approx n$, the complexity is roughly $O(nk \cdot |C_{i,j}| + n \cdot |\pi_{FDKG_i}|)$. In our BabyJub/Groth16 instantiation, this leads to $O(nk)$ complexity dominated by encrypted shares.
- **Round 2 (Online Reconstruction):** Each online party $P_i \in \mathbb{T}$ might broadcast its partial secret d_i (1 scalar) and proof π_{PS_i} , plus potentially many decrypted shares $s_{j,i}$ (1 scalar each) and proofs $\pi_{SPS_{j,i}}$. Let m_i be the number of shares party P_i reveals (as a guardian for others). The complexity is roughly $O(|\mathbb{T}| \cdot (|\mathbb{Z}_q| + |\pi_{PS_i}| + m_{\max} \cdot (|\mathbb{Z}_q| + |\pi_{SPS_{j,i}}|)))$, where $m_{\max} = \max_i m_i$. In the worst case, where $|\mathbb{T}| \approx n$ and $m_{\max} \approx n$, this can approach $O(n^2)$.

FDKG offers $O(n \cdot k)$ complexity for distribution, which is similar to standard DKG if $k \approx n$. Reconstruction complexity varies but can reach $O(n^2)$ in worst-case guardian—when a single party is selected as a guardian by every other participant and must reveal all $n - 1$ shares.

D. FDKG Example

Consider $n = 10$ potential parties $\{P_1, \dots, P_{10}\}$, local threshold $t = 2$, and guardian set size $k = 3$. Suppose $\mathbb{D} = \{P_1, P_3, P_5, P_7, P_9\}$ participate correctly in Round 1.

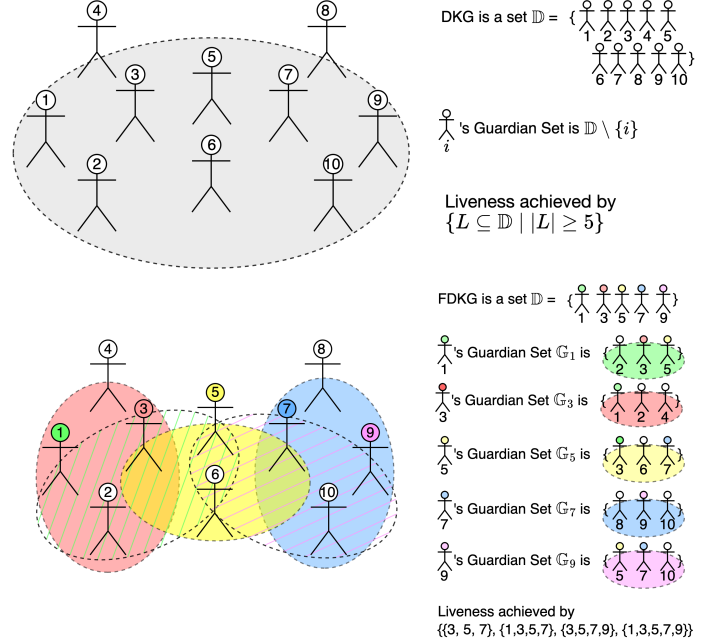


Figure 2: Visual comparison of $(k = 10, t = 5)$ -DKG protocol (above) and $(n = 10, k = 3, t = 2)$ -FDKG (below)

• Round 1 Online

- P_1 chooses $\mathcal{G}_1 = \{P_2, P_3, P_5\}$.
- Generates (d_1, E_1) .
- Shares d_1 into $s_{1,2}, s_{1,3}, s_{1,5}$.
- Encrypts shares to get $\mathbb{C}_1 = \{C_{1,2}, C_{1,3}, C_{1,5}\}$.
- Generates π_{FDKG_1} and broadcasts. Others in $\mathbb{D}$ do similarly.

• Round 1 Offline

- Once broadcasts are received, everyone computes $\mathbf{E} = \prod_{P_i \in \mathbb{D}} E_i$.

• Round 2 Online

- Suppose $\mathbb{T} = \{P_3, P_5, P_7\}$.
- P_3, P_5, P_7 each broadcast their $(\text{secret}, d_i, \pi_{PS_i})$.
- As guardians, they also decrypt and broadcast the necessary shares for offline parties:
 - * P_3 and P_5 (guardians for P_1) broadcast $(\text{share}, P_1, s_{1,3}, \pi_{SPS_{1,3}})$ and $(\text{share}, P_1, s_{1,5}, \pi_{SPS_{1,5}})$ respectively.
 - * P_5 and P_7 (guardians for P_9) broadcast $(\text{share}, P_9, s_{9,5}, \pi_{SPS_{9,5}})$ and $(\text{share}, P_9, s_{9,7}, \pi_{SPS_{9,7}})$ respectively.

• Round 2 Offline

- Collect all valid broadcasts: direct secrets from P_3, P_5, P_7 and guardian shares for P_1 and P_9 .
- Reconstruct offline parties using $t = 2$ shares:
 - * For P_1 : use $\{(P_3, s_{1,3}), (P_5, s_{1,5})\}$ to compute $d'_1 \leftarrow \text{Reconstruct}(\{P_3, P_5\}, \{s_{1,3}, s_{1,5}\})$ and verify $G^{d'_1} = E_1$.

- * For P_9 : use $\{(P_5, s_{9,5}), (P_7, s_{9,7})\}$ to compute $d'_9 \leftarrow \text{Reconstruct}(\{P_5, P_7\}, \{s_{9,5}, s_{9,7}\})$ and verify $G^{d'_9} = E_9$.
- With d_3, d_5, d_7 revealed directly and d'_1, d'_9 reconstructed, compute $\mathbf{d} = \sum_{P_i \in \mathbb{D}} d_i$.

V. SECURITY ANALYSIS

This section analyzes the security of Π_{FDKG} in two phases: *Generation* (Round 1) and *Reconstruction* (Round 2). We adopt the standard real/ideal paradigm [49, 50]. A detailed ideal functionality $\mathcal{F}_{FDKG}$ appears in Appendix A.

A. Security Model and Assumptions

- 1) **Parties and communication.** There are n potential parties $\mathbb{P} = \{P_1, \dots, P_n\}$. Communication occurs over an authenticated public broadcast channel in *synchronous* rounds.
- 2) **Adversary.** Unless stated otherwise, the adversary is *static* probabilistic polynomial time (PPT) and corrupts a fixed subset $\mathcal{C} \subset \mathbb{P}$ before the protocol starts; corrupted parties are fully Byzantine. Let $\mathcal{H} = \mathbb{P} \setminus \mathcal{C}$ denote the honest set. Honest parties may be unavailable in Round 2 (crash/absence), which affects liveness. Section VI-C discusses adaptive adversaries and targeted attacks.
- 3) **Cryptographic assumptions.**
 - **PKE security.** The channel for encrypted shares (Enc, Dec) (Section III-F) is assumed *IND-CPA* secure. In our implementation this is an ElGamal variant over the BabyJub subgroup. Security reduces to the *DDH* assumption (and ultimately DLP hardness) in this group; ciphertext malleability is acceptable because correctness is enforced by NIZK soundness. Any alternative IND-CPA PKE could be substituted.
 - **NIZK security.** The proof system (Setup, Prove, Verify) (Section III-E) is assumed computationally sound and zero-knowledge in the CRS model. We instantiate Groth16 over the BN254 pairing curve (also known as bn128, the default in `circom/snarkjs`); this uses a circuit-specific structured reference string (SRS) generated in a trusted setup. Soundness relies on standard pairing-group assumptions on BN254; zero-knowledge follows from the Groth16 simulator. Other systems (e.g., PLONK/Marlin with a universal updatable SRS [43, 44], or STARKs with transparent setup [45]) are compatible at the level of assumptions used here, trading proof size/proving time for setup properties and security level.
 - **Discrete logarithm hardness.** We assume DLP hardness in (i) the BabyJub subgroup used for ElGamal and (ii) the pairing groups induced by the chosen SNARK curve, as applicable.

B. Security of the Generation Phase (Round 1)

We prove Generation-phase security in the real/ideal paradigm using the ideal functionality $\mathcal{F}_{FDKG}$ (Appendix A). Informally, the simulator replaces honest encrypted shares by encryptions of 0 and uses the NIZK simulator to generate proofs; IND-CPA and zero-knowledge imply indistinguishability, while NIZK soundness enforces well-formedness. The following theorem summarizes the resulting properties.

Theorem 1 (Generation: Correctness, Privacy, Robustness). Assume the PKE is IND-CPA and the NIZK is zero-knowledge and computationally sound. For the set of valid participants $\mathbb{D}$ determined by verification:

- 1) **Correctness.** For each $i \in \mathbb{D}$, $E_i = G^{d_i}$ and

$$\mathbf{E} = \prod_{i \in \mathbb{D}} E_i = G^{\sum_{i \in \mathbb{D}} d_i}.$$

Moreover, every published $C_{i,j}$ encrypts $f_i(j)$ for a degree- $(t-1)$ polynomial f_i with $f_i(0) = d_i$.

- 2) **Privacy (during Generation).** The adversary's view leaks no information about $\{d_i\}_{i \in \mathbb{D} \cap (\mathbb{P} \setminus \mathcal{C})}$ beyond $(\mathbf{E}, \{E_i\}_{i \in \mathbb{D}})$ and the shares explicitly addressed to corrupted recipients $\{[d_i]_j \mid j \in \mathcal{G}_i \cap \mathcal{C}\}$.
- 3) **Robustness (completion).** Malformed broadcasts (invalid proofs) are rejected; corrupted parties cannot force incorrect acceptance. Honest parties with valid broadcasts are included in $\mathbb{D}$ and obtain their outputs as per $\mathcal{F}_{FDKG}$.

Proof sketch. Simulate honest parties by encrypting 0 and using simulated NIZKs; IND-CPA and zero-knowledge yield indistinguishability. NIZK soundness binds each dealer's ciphertexts to a single polynomial and E_i , ensuring well-formedness. Verification excludes malformed data, implying robustness. $\square$

C. Security of the Reconstruction Phase (Round 2)

This subsection assumes the Generation phase completed successfully (Theorem 1) and that the Round 2 NIZKs (π_{PS_i} , $\pi_{SPS_{i,j}}$) are computationally sound. We analyze *Correctness*, *Privacy (reconstruction)*, and *Liveness*.

a) *Notation for Reconstruction:* Define the family of reconstruction-capable sets

$$R := \left\{ S \subseteq \mathbb{P} \mid \forall i \in \mathbb{D} : i \in S \vee |S \cap \mathcal{G}_i| \geq t \right\}. \quad (1)$$

Intuitively, $S \in R$ iff, for every participant i , S contains either i or at least t of i 's guardians. Round 2 privacy and liveness depend on the guardian topology $\{\mathcal{G}_i\}$ via R rather than on a single global corruption fraction, as in traditional uniform DKGs.

Theorem 2 (Reconstruction: Correctness). If for every $i \in \mathbb{D}$ either (i) there is a valid broadcast $(P_i, \text{secret}, d_i, \pi_{PS_i})$, or (ii) there exist t distinct guardians $j \in \mathcal{G}_i$ with valid broadcasts $(P_j, \text{share}, P_i, s_{i,j}, \pi_{SPS_{i,j}})$, then any observer reconstructs the unique $\mathbf{d} = \sum_{i \in \mathbb{D}} d_i$. Moreover, checking $G^{d_i} = E_i$ prevents substitution.

Proof sketch. By SSS, any t correct shares interpolate $f_i(0) = d_i$. Soundness of π_{PS_i} and $\pi_{SPS_{i,j}}$ ensures that only values consistent with Round-1 commitments are accepted. Summing over i yields $\mathbf{d}$. $\square$

Theorem 3 (Reconstruction: Privacy). The adversary can reconstruct the global secret if and only if there exists a set $S \subseteq \mathcal{C}$ such that $S \in R$. Equivalently, *reconstruction privacy holds* iff $\mathcal{C} \notin R$.

Proof sketch. If the adversary reconstructs, it must possess, for each i , either i 's partial secret (when $i \in \mathcal{C}$) or t decryptable guardian shares (when t guardians are corrupted). Collect those principals into S ; then $S \subseteq \mathcal{C}$ and $S \in R$.

Conversely, if $S \subseteq \mathcal{C}$ and $S \in R$, then for every i the adversary controls either i or t guardians. In the first case it knows d_i ; in the second it decrypts t shares and reconstructs d_i . Summing yields $\mathbf{d}$. $\square$

Theorem 4 (Reconstruction: Liveness). The adversary cannot prevent reconstruction if and only if there exists a set $S \in R$ such that $S \subseteq U \setminus \mathcal{C}$. Equivalently,

$$\forall i \in \mathbb{D} : (i \notin \mathcal{C}) \vee (|\mathcal{C} \cap \mathcal{G}_i| \leq k - t).$$

Proof sketch. If such S exists, then S contains, for each i , either i or t honest guardians; reconstruction proceeds as in Theorem 2.

Conversely, if no such S exists, then for some i the adversary controls i and at least $k - t + 1$ of $\mathcal{G}_i$, preventing both direct revelation and a t -guardian reconstruction path; hence liveness fails. $\square$

Potential attacks and mitigations: *Share withholding / selective opening* in Round 2 is captured by Theorem 4: liveness fails for i only if the adversary both corrupts i and at least $k - t + 1$ of $\mathcal{G}_i$. *Forgery or equivocation* is ruled out by soundness of π_{PS_i} and $\pi_{SPS_{i,j}}$. *Guardian concentration attacks* (attracting many delegations to a small set of guardians) increase redundancy under random churn but reduce privacy thresholds under adaptivity (Theorem 3); deployers can mitigate by (i) capping per-guardian incoming delegations, (ii) diversifying recommendation lists, (iii) sampling a fraction of guardians via a verifiable random function (VRF), and (iv) rate-limiting or staking for high-degree guardians. Finally, *Sybil guardians* are out of scope of our PKI model but can be addressed operationally via admission controls (identity or stake).

On topology dependence: Two extreme cases illustrate the role of overlap:

- *Disjoint guardians.* If $\mathcal{G}_i \cap \mathcal{G}_{i'} = \emptyset$ for $i \neq i'$, then making a fixed i unrecoverable requires at least $1 + (k - t + 1)$ targeted corruptions (participant i and $k - t + 1$ distinct guardians).
- *Full overlap.* If $\mathcal{G}_i = \mathcal{G}$ for all i , then corrupting $k - t + 1$ parties in $\mathcal{G}$ makes every corrupted participant i unrecoverable. Thus, the fraction of corruptions needed to break liveness or privacy depends on $\{\mathcal{G}_i\}$.

Next section empirically studies overlap policies (Random vs. Barabási–Albert) and their impact on success rates.

VI. LIVENESS SIMULATIONS

This section empirically evaluates *liveness* under non-adversarial (random) churn. The goal is to estimate, for a configuration (n, p, r, k, t) and a guardian-selection policy, the probability that the Round 2 reconstruction predicate induced by (1) is satisfied. These experiments complement the conditions in Theorems 4 and 3: they do not constitute a cryptographic proof, but quantify the operational regimes where reconstruction is likely to succeed in the presence of availability failures.

Evaluation predicate: Let $\mathbb{D}$ be the set of valid Round 1 participants and $\mathbb{T} \subseteq \mathbb{P}$ be the Round 2 actors that are present. A trial is a *success* iff

$$\forall i \in \mathbb{D} : (i \in \mathbb{T}) \vee (|\mathbb{T} \cap \mathcal{G}_i| \geq t), \quad (2)$$

i.e., iff $\mathbb{T} \in R$ with R as in (1); see Theorem 4.

A. Experimental Design and Rationale

Topology policies (guardian selection): We study two canonical policies that bracket real deployments:

- **Random (Erdős–Rényi, ER):** guardians are chosen uniformly; this models hub-free environments (uniform lists, randomized assignment).
- **Barabási–Albert (BA):** guardians are chosen with preferential attachment; this models environments with socially/operationally prominent “hubs” (recognizable, responsive, institutionally visible parties).

Both policies enforce $|\mathcal{G}_i| = k$ with local threshold $t \leq k$. ER provides a neutral baseline without hubs; BA captures heavy-tailed guardian popularity, which concentrates redundancy around hubs and (under random churn) tends to increase the number of disjoint reconstruction paths.

Parameters and sampling: We explore $n \in \{50, 100, 200, \dots, 1000\}$, $p \in \{0.1, 0.2, \dots, 1.0\}$, $r \in \{0.1, 0.2, \dots, 1.0\}$, $k \in \{1, 3, \dots, n - 1\}$, and $t \in \{1, \dots, k\}$. Round 1 participation draws $\mathbb{D}$ with $|\mathbb{D}| \approx pn$. Round 2 availability draws $\mathbb{T}$ with expected size $\approx r|\mathbb{P}|$. Unless stated otherwise, we run 100 independent, identically distributed (i.i.d.) trials per configuration and report the *success rate* (fraction of trials satisfying (2)).

DKG vs FDKG: Classical (t, n) -DKG is the special case of FDKG with full participation and maximal guardians:

$$\text{DKG}(t, n) \equiv \text{FDKG}(p=1, t, k=n-1).$$

Therefore, any comparison at identical parameters is tautological. In figures we include a *reference line* labelled “DKG” defined as the FDKG curve at $(p=1, k=n-1)$; all other curves are FDKG with $(p \leq 1, k \leq n-1)$. This makes explicit that differences arise from configuration, not from different protocols.

Implementation and artifacts: We use the public simulator at <https://fdkg.stan.bar> (source: <https://github.com/stanbar/fdkg>). Figure 3 illustrates ER vs. BA for the same (n, p, r, k, t) .

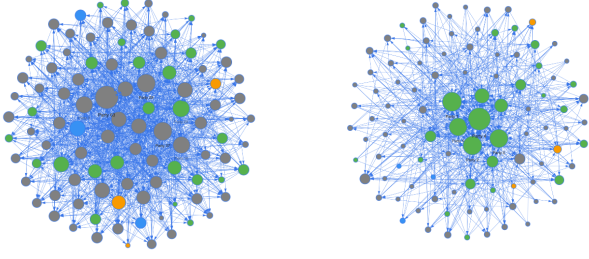


Figure 3: ER (left) vs. BA (right) guardian-selection examples for $n = 100, p = 0.3, r = 0.9, k = 5, t = 3$. Gray: absent; green: present in both rounds; blue: Round 1 only; orange: Round 2 only.

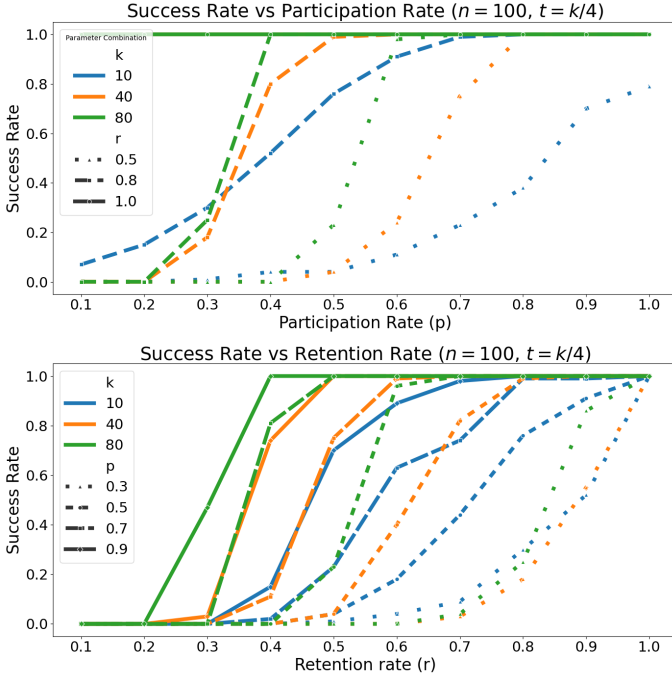


Figure 4: Success vs. participation p (top) and retention r (bottom) at $n = 100$ with $t = k/4$. Line styles encode the other axis; colors encode k .

B. Results and Interpretation

Retention dominates: Retention r is the primary driver of liveness. With $r \approx 0.8$, high success is attainable even for modest k at partial participation. For instance, for $n = 100, p = 0.7, k = 10$ achieves high success. Increasing $k = 40$ yields near-certainty even at $p = 0.5$ (Figure 4).

With $r \approx 0.5$, approaching certainty requires both higher p and larger k (e.g., for $n = 100, p \gtrsim 0.7$ and $k \gtrsim 40$) (Figure 4).

Threshold vs. redundancy: For fixed (n, p, r, k) , success decreases as t/k increases. Empirically, for $n = 100, p = 0.8, r = 0.9$, liveness remains robust up to roughly $t \lesssim 0.7k$; with $r = 0.5$ robustness holds up to roughly $t \lesssim 0.4k$ (Figure 5).

Topology under random churn: BA generally outperforms ER for small/medium n because preferential attachment concentrates redundancy around hubs, creating multiple

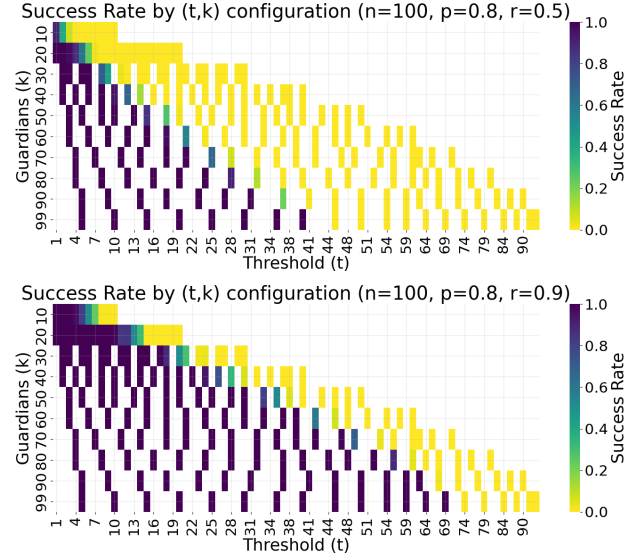


Figure 5: Success over (k, t) for $n = 100, p = 0.8$, with $r = 0.5$ (top) and $r = 0.9$ (bottom). Darker indicates higher success. Higher r expands the feasible region (larger t at fixed k).

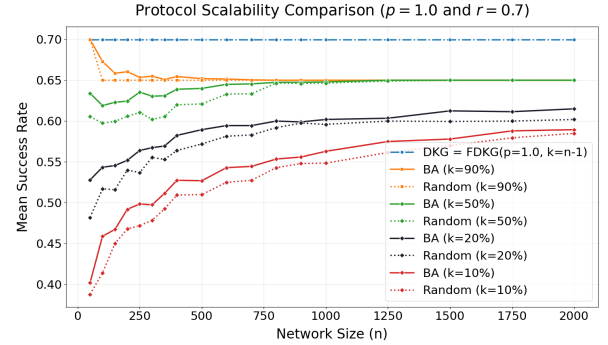


Figure 6: Scalability vs. guardian policy for $p = 1.0, r = 0.7$. The DKG line is the FDKG configuration ($p=1, k=n-1$) (classical (t, n) -DKG). BA (solid) generally exceeds ER (dashed) under random churn; the gap narrows with larger n .

disjoint reconstruction paths. As n grows, the gap narrows (Figure 6). This advantage pertains to *random* availability failures and does not imply robustness to targeted hub failures.

Scaling envelope: Across n , the DKG line (FDKG at $p=1, k=n-1$) acts as an upper envelope at a given r . FDKG approaches this envelope as k increases; for smaller k , it trades communication cost for liveness probability in a topology- and r -dependent manner.

C. Adaptive Adversaries

The security analysis (Section V) characterizes reconstruction privacy via R (Theorem 3) and liveness via the existence of an honest $S \in R$ (Theorem 4). Under *adaptive* corruption at Round 2, two objectives diverge:

Targeting liveness: To falsify (2) for i , one must simultaneously prevent direct revelation (corrupt i) and reduce the remaining honest guardians below t (thus corrupt $\geq k-t+1$ guardians).

Targeting privacy: Privacy fails iff the corrupted set contains a reconstruction-capable set for all participants' guardian requirements (Theorem 3), i.e., there exists $S \subseteq \mathcal{C}$ with $S \in R$. The attacker needs to control enough guardians to reconstruct every participant's partial secret (unless that participant is directly corrupted). How many corruptions are needed depends mainly on how guardian sets overlap:

- *Full overlap* ($G_i = G$ for all i): corrupt any t in G to reconstruct *all* partial secrets (recovers the classical DKG threshold).
- *Disjoint guardians* ($G_i \cap G_j = \emptyset$): the adversary must, for each i , either corrupt i or t members of G_i , so the optimum scales with the number of participants unless many i are directly corrupted.

Thus, diverse guardian choices improve availability under random churn (Section VI-B), but privacy under targeted attacks depends on guardian-set overlaps. Operationally, implementations should provide UIs that encourage honest parties to choose diverse guardian sets to avoid over-concentration.

VII. APPLICATION: FDKG-ENHANCED VOTING PROTOCOL

In this section, we demonstrate an application of FDKG within an internet voting protocol. This protocol builds upon the foundational three-round structure common in threshold cryptosystems [51], incorporates multi-candidate vote encoding [52], and crucially utilizes FDKG (Section IV) for decentralized and robust key management.

The core idea is to use FDKG to establish a threshold ElGamal public key $\mathbf{E}$ where the corresponding secret key $\mathbf{d}$ is distributed among participants and their guardians, mitigating single points of failure and handling dynamic participation.

The voting protocol consists of three primary rounds:

- 1) **Key Generation (FDKG):** Participants establish the joint public encryption key $\mathbf{E}$ and distribute shares of their partial decryption keys.
- 2) **Vote Casting:** Voters encrypt their choices under $\mathbf{E}$ and submit their ballots with proofs of validity.
- 3) **Tallying:** Participants collaboratively decrypt the sum of encrypted votes to reveal the final tally.

A. Round 1: Federated Distributed Key Generation

This round follows the Π_{FDKG} protocol exactly as described in Section IV-B1.

- **Participation:** Optional for any party $P_i \in \mathbb{P}$.
- **Actions:** Each participating party P_i selects its guardian set $\mathcal{G}_i$, generates its partial key pair (d_i, E_i) , computes and encrypts shares $C_{i,j}$ for its guardians, generates a NIZK proof π_{FDKG_i} , and broadcasts its contribution.
- **Outcome:** Observers verify the proofs π_{FDKG_i} to determine the set of valid participants $\mathbb{D}$. The global public encryption key for the election is computed as $\mathbf{E} = \prod_{P_i \in \mathbb{D}} E_i$. The public state includes $\mathbb{D}$, $\mathbf{E}$, and the broadcasted tuples $\{E_i, \mathcal{G}_i, \{C_{i,j}\}, \pi_{FDKG_i}\}_{P_i \in \mathbb{D}}$.

B. Round 2: Vote Casting

Let $\mathbb{V} \subseteq \mathbb{P}$ be the set of eligible voters participating in this round. For each voter $P_i \in \mathbb{V}$:

- 1) **Vote Encoding:** Encode the chosen candidate into a scalar value $v_i \in \mathbb{Z}_q$. We use the power-of-2 encoding from [53]:

$$v_i = \begin{cases} G^{2^0} & \text{for candidate 1} \\ G^{2^m} & \text{for candidate 2} \\ \vdots & \vdots \\ G^{2^{(c-1)m}} & \text{for candidate } c \end{cases}$$

where m is the smallest integer s.t. $2^m > n$, and c is the number of candidates. The vote is represented as a point on the curve.

- 2) **Vote Encryption (ElGamal):** Generate a random blinding factor $r_i \leftarrow \mathbb{Z}_q$. Compute the encrypted ballot $B_i = (C1_i, C2_i)$ where:

$$C1_i = G^{r_i}$$

$$C2_i = \mathbf{E}^{r_i} \cdot v_i \quad (\text{using group operation in } G)$$

- 3) **Proof Generation:** Generate a NIZK proof π_{Ballot_i} demonstrating that B_i is a valid ElGamal encryption of a vote v_i from the allowed set $\{G^{2^0}, G^{2^m}, \dots, G^{2^{(c-1)m}}\}$ under public key $\mathbf{E}$, without revealing v_i or r_i . (See Appendix F).
- 4) **Broadcast:** Broadcast the ballot and proof (B_i, π_{Ballot_i}) .

State after Voting: The broadcast channel is appended with the set of valid ballots and proofs:

- $\{(B_i, \pi_{Ballot_i}) \mid P_i \in \mathbb{V} \text{ and } \pi_{Ballot_i} \text{ is valid}\}$. Let $\mathbb{V}' \subseteq \mathbb{V}$ be the set of voters whose ballots verified.

C. Round 3: Tallying

This round involves threshold decryption of the combined ballots. It follows the structure of the FDKG Reconstruction Phase (Section IV-B2).

- 1) *Online Tally:* A subset of parties $\mathbb{T} \subseteq \mathbb{P}$ participate. For each party $P_i \in \mathbb{T}$:

- 1) **Compute Homomorphic Sum:** Aggregate the first components of all valid ballots: $C1 = \prod_{P_j \in \mathbb{V}'} C1_j$.
- 2) **Provide Partial Decryption (if applicable):** If $P_i \in \mathbb{D}$:
 - Compute the partial decryption: $PD_i = (C1)^{d_i}$.
 - Generate a NIZK proof π_{PD_i} proving that PD_i was computed correctly using the secret d_i corresponding to the public E_i . (See Appendix G).
 - Broadcast $(P_i, \text{pdecrypt}, PD_i, \pi_{PD_i})$.

- 3) **Provide Decrypted Shares:** For each encrypted share $C_{j,i}$ received in Round 1 (where $P_j \in \mathbb{D}$ and $P_i \in \mathcal{G}_j$):

- Decrypt the share: $s_{j,i} = \text{Dec}_{sk_i}(C_{j,i})$.
- Compute the share's contribution to decryption: $[PD_j]_i = (C1)^{s_{j,i}}$.
- Generate a NIZK proof $\pi_{PDS_{j,i}}$ proving correct decryption of $C_{j,i}$ to $s_{j,i}$ and correct exponentiation of $C1$ by $s_{j,i}$. (See Appendix H).
- Broadcast $(P_i, \text{pdecryptshare}, P_j, [PD_j]_i, \pi_{PDS_{j,i}})$.

State after Online Tally: The broadcast channel contains potentially revealed partial decryptions (PD_i, π_{PD_i}) and shares of partial decryptions $([PD_j]_i, \pi_{PDS_{j,i}})$ from parties in $\mathbb{T}$.

2) *Offline Tally:* Performable by any observer using publicly available data.

- 1) **Aggregate Ballot Components:** Compute the aggregated components of valid ballots:

$$C1 = \prod_{P_k \in \mathbb{V}'} C1_k$$

$$C2 = \prod_{P_k \in \mathbb{V}'} C2_k$$

- 2) **Reconstruct Partial Decryptions:** For each $P_i \in \mathbb{D}$:

- Try to find a valid broadcast $(P_i, \text{pdecrypt}, PD_i, \pi_{PD_i})$. If found and π_{PD_i} verifies, use $PD_i = PD_i$.
- Otherwise, collect all valid broadcasted shares $(P_k, \text{pdecryptshare}, P_i, [PD_i]_j, \pi_{PDS_{i,j}})$ where $P_k = P_j$ and $\pi_{PDS_{i,j}}$ verifies. Let I_i be the set of indices j for which valid shares were found.
- If $|I_i| \geq t$: Select $I'_i \subseteq I_i$, $|I'_i| = t$. Use Lagrange interpolation in the exponent to compute $PD_i = \prod_{P_j \in I'_i} ([PD_i]_j)^{\lambda_j}$. Verify this reconstructed value against E_i .
- If neither direct secret nor enough shares are available/valid, the tally fails.

- 3) **Combine Partial Decryptions:** Compute the combined decryption factor Z :

$$Z = \prod_{P_i \in \mathbb{D}} PD_i = (C1)^{\sum d_i} = (C1)^d$$

- 4) **Decrypt Final Result:** Compute the product of the encoded votes:

$$M = C2 \cdot Z^{-1} = \left(\prod_{P_k \in \mathbb{V}'} E^{r_i} \cdot v_k \right) \cdot \left(\prod_{P_k \in \mathbb{V}'} G^{r_i} \right)^{-d}$$

Since $E = G^d$, this simplifies to:

$$M = \prod_{P_k \in \mathbb{V}'} v_k = G^{\sum x_k 2^{(k-1)m}}$$

where x_k is the number of votes for candidate k .

- 5) **Extract Counts:** Solve the discrete logarithm problem $M = G^{\text{result}}$. Since ‘result’ = $\sum x_k 2^{(k-1)m}$ and the total number of votes $|\mathbb{V}'|$ is known (bounding the x_k), the counts x_k can be extracted efficiently, e.g., using Pollard’s Rho or Baby-Step Giant-Step for the small resulting exponent, or simply by examining the binary representation if m is chosen appropriately [52].

VIII. PERFORMANCE EVALUATION

This section presents a performance analysis of the FDKG-enhanced voting protocol described in Section VII. We evaluated computational costs (NIZK proof generation times and offline tally time) and communication costs (total broadcast message sizes). All NIZK evaluations use Groth16 [42] as the

NIZK proof system, assuming a proof size of 256 bytes (uncompressed). We briefly investigated using the PLONK [43] proof system as an alternative potentially avoiding a trusted setup, but found it computationally infeasible for our circuits; for instance, generating a π_{FDKG_i} proof with parameters $(t = 3, k = 10)$ took over 8 minutes with PLONK, and larger configurations exceeded available memory resources. Therefore, we proceeded with Groth16.

a) *Computational and Communication Costs per Message Type:* Table III summarizes the core performance metrics for the different messages broadcast during the online phases of the protocol. It includes the time to generate the required NIZK proof and the total message size, which comprises the cryptographic payload plus the 256-byte Groth16 proof.

The online computational cost is dominated by the generation of the π_{FDKG_i} proof, especially as t and k increase. The communication cost for the FDKG Generation phase is primarily driven by k . Messages related to voting and tallying are significantly smaller individually, but the total communication cost of tallying depends heavily on how many secrets and shares need to be revealed publicly.

b) *Offline Tally: Vote Extraction Cost:* The final step of the Offline Tally phase (Section VII, Offline Tally, Step 5) involves extracting the individual vote counts $\{x_k\}$ from the aggregated result $M = G^{\sum x_k 2^{(k-1)m}}$. This requires solving a Discrete Logarithm Problem (DLP) where the exponent has a specific known structure. Since the maximum value of the exponent is bounded by the total number of valid votes $|\mathbb{V}|$, standard algorithms like Baby-Step Giant-Step or Pollard’s Rho can be used.

We measured the time required for this vote extraction step for varying numbers of voters $|\mathbb{V}|$ and candidates c . The results are presented in Figure 7. The data shows that the time required grows approximately linearly with the number of voters (for a fixed number of candidates) but grows exponentially with the number of candidates c . For instance, with 2 candidates, extracting results for up to 1000 voters takes only milliseconds. However, with 5 candidates, extracting the result for 100 voters took over 100 seconds.

This indicates that while the FDKG and online tallying phases scale well, the vote extraction step using this specific power-of-2 encoding [53] imposes a practical limit on the number of candidates that can be efficiently supported, especially when combined with a large number of voters.

c) *Example Scenario Calculation:* Let’s estimate the total broadcast communication size for a specific scenario:

- $n = 100$ potential participants.
- $|\mathbb{D}| = 50$ valid participants in Round 1 (FDKG Generation).
- Each participant uses $k = 40$ guardians.
- $|\mathbb{V}| = 50$ valid ballots cast in Round 2.
- In Round 3 (Online Tally), assume $|\mathbb{T} \cap \mathbb{D}| = 40$ participants reveal their secrets directly.
- Assume for reconstruction, each of the 40 online talliers reveals, on average, 40 shares, this leads to approximately $40 \times 40 = 1600$ total shares being revealed (this is a simplification, the actual number depends on guardian set overlaps and the specific set of online talliers).

Table III: Performance Summary: NIZK Proving Time and Total Message Size per Broadcast Message Type.

Measurement	FDKG Generation			Ballot Broadcast	Partial Dec. Broadcast	Part. Dec. Share Broadcast
	(t=3, k=10)	(t=10, k=30)	(t=30, k=100)			
NIZK Proving Time	3.358 s	4.415 s	14.786 s	0.211 s	0.619 s	0.580 s/share
Base Payload Size	1664 B	4864 B	16064 B	128 B	64 B	64 B/share
NIZK Proof Size	256 B	256 B	256 B	256 B	256 B	256 B/share
Total Message Size (Bytes)	1920	5120	16320	384	320	320/share

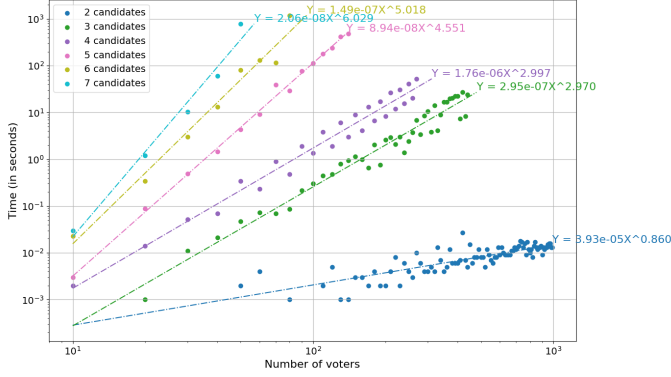


Figure 7: Time required to solve DLP for vote extraction with respect to the number of candidates and number of voters (log-log scale).

Using the "Total Message Size (Bytes)" figures from Table III:

- 1) **FDKG Dist.:** $50 \times (64 + 40 \times 160 + 256) = 50 \times 6720 \text{ B} = 336000 \text{ B} (336.0 \text{ kB})$.
- 2) **Voting:** $50 \times 384 \text{ B} = 19200 \text{ B} (19.2 \text{ kB})$.
- 3) **Tallying (Online):**
 - Secrets: $40 \times 320 \text{ B} = 12800 \text{ B} (12.8 \text{ kB})$.
 - Shares: $1600 \times 320 \text{ B} = 512000 \text{ B} (512.0 \text{ kB})$.
- 4) **Total Broadcast:** $336000 + 19200 + 12800 + 512000 = 880000 \text{ B} (880.0 \text{ kB})$.

The offline tally time for this scenario (50 voters) would be milliseconds for 2-4 candidates, but could increase to several minutes for 5 or more candidates, as indicated by Figure 7.

Large-scale scenario ($n = 5000$): Assume $|\mathbb{D}| = 2500$, $k = 40$, $|\mathbb{V}| = 2500$, $|\mathbb{T} \cap \mathbb{D}| = 2000$, and 40 shares revealed per tallier (80,000 shares total). Using Table III sizes:

$$\begin{aligned}
 \text{FDKG Dist.} &= 2500 \times 6720 = 16,800,000 \text{ B} (16.8 \text{ MB}), \\
 \text{Voting} &= 2500 \times 384 = 960,000 \text{ B} (0.96 \text{ MB}), \\
 \text{Tally (secrets)} &= 2000 \times 320 = 640,000 \text{ B} (0.64 \text{ MB}), \\
 \text{Tally (shares)} &= 80,000 \times 320 = 25,600,000 \text{ B} (25.6 \text{ MB}), \\
 \text{Total} &= 44,000,000 \text{ B} (44.0 \text{ MB}).
 \end{aligned}$$

Communication is dominated by share revelations; for fixed k and average shares per tallier, growth is near-linear in $|\mathbb{D}|$.

d) Scalability and load balance: FDKG uses one broadcast round in Generation and one in Reconstruction. End-to-end latency is dominated by (i) local proving time for π_{FDKG_i} and (ii) network propagation per round; Reconstruction latency additionally depends on the number of shares each online guardian reveals and proves. Load is uneven by design: some guardians receive more delegations and therefore handle more

decryption-share work. Let c_g denote the number of parties that selected guardian g (guardian "popularity"). A simple way to summarize skew is to report $\max_g c_g$, the median of $\{c_g\}$, and the ratio $\max_g c_g / \text{median}(\{c_g\})$. In our experiments, this ratio is larger under BA than ER (reflecting hubs). Operators can bound peak load by capping per-guardian assignments, randomizing a fraction of guardian picks, or recommending diversified guardian sets.

The Generation broadcast cost scales as $\mathcal{O}(n \cdot k)$ elements (Section IV); with fixed k the total cost is linear in n . Reconstruction can reach $\mathcal{O}(n^2)$ in worst-case guardian topologies (some parties chosen by all other participants), but in practice remains near $\mathcal{O}(n \cdot k)$ when overlap is moderate. Hierarchical deployments (partitioning parties into domains, then aggregating domain-level partial keys) can reduce the maximum number of counterparties any single party must send to or serve during Generation/Reconstruction, while preserving heterogeneous trust.

IX. DEPLOYMENTS

The FDKG protocol (and FDKG-based voting application), requiring only a shared communication space (message board) where participants can publicly post and read messages, is adaptable to various deployment environments. We consider three illustrative scenarios:

- 1) **Public Blockchains:** The protocol can be implemented on public blockchains. For instance, interactions could be managed via smart contracts. Features like ERC-4337 (Account Abstraction) [54] could further simplify deployment by enabling centralized payment of transaction fees, potentially lowering adoption barriers for participants.
- 2) **Peer-to-peer Networks:** The protocol is inherently suitable for peer-to-peer (P2P) networks. Technologies such as Wesh Network¹, which provide decentralized communication infrastructure, can serve as a foundation for FDKG deployments.
- 3) **Centralized Messengers:** For ease of deployment and accessibility, FDKG could be layered on top of existing centralized messenger platforms (e.g., Telegram, Signal, WhatsApp), where a group chat acts as the message board. However, it is important to note that the centralized nature of the messenger could introduce a trusted third party capable of censorship, potentially impacting protocol liveness or fairness if it selectively blocks messages.

Figure 8 visually depicts these potential deployment architectures.

¹Wesh Network, an asynchronous mesh network protocol by Bertly Technologies, <https://wesh.network/>

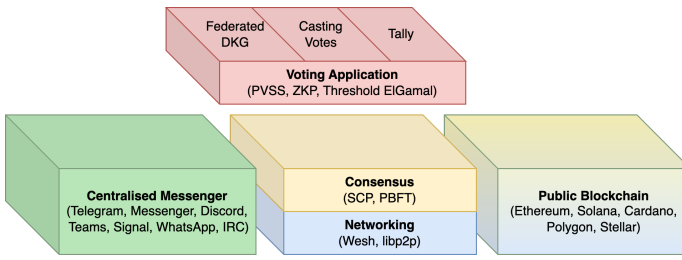


Figure 8: Three possible deployments of the protocol: Centralised messenger, ad-hoc peer-to-peer network, and public blockchain.

X. LIMITATIONS AND FUTURE WORK

While FDKG offers enhanced flexibility and resilience to dynamic participation, our analysis and implementation highlight areas for further improvement and research.

Cryptographic Overhead: The protocol inherits computational overhead from its cryptographic building blocks, particularly NIZK proofs. Although Groth16 proofs, as used in our implementation, are efficient for mid-scale settings (e.g., tens to a few hundred participants), very large-scale or time-critical applications (like high-throughput elections) might necessitate faster proving systems (e.g., different SNARKs or STARKs) or techniques for proof aggregation to reduce verification load.

Offline Tallying Scalability: As discussed in Section VIII, the offline tallying step in our FDKG-enhanced voting example, specifically the brute-force decoding of the final aggregated vote via DLP solution, exhibits exponential scaling with the number of candidates. Developing more efficient multi-candidate vote encoding schemes or advanced number-theoretic techniques for vote extraction remains an important open research challenge for such applications.

Usability and guardian selection: Non-experts may find manual guardian selection burdensome. In practice, UIs can (i) offer curated recommendations that blend centrality (availability) with diversity (to avoid over-concentration), (ii) enforce soft caps on per-guardian assignments, (iii) set (k, t) defaults from observed retention (e.g., $t \approx 0.3$ – 0.5 of k for moderate r), and (iv) warn about privacy risks under highly overlapping guardian choices. These controls preserve FDKG’s heterogeneous trust while reducing user burden.

Trusted Setup for NIZKs: Our current NIZK instantiation (Groth16) requires a trusted setup for its Common Reference String (CRS). While this is a one-time cost per circuit, it represents a potential point of centralization or trust. Future work could explore integrating FDKG with NIZK systems that have transparent or universal setups (e.g., STARKs, or SNARKs like PLONK with a universal updatable SRS) to mitigate this dependency, although this may involve performance trade-offs.

Resharing and forward secrecy: FDKG, as presented, is a bootstrapping primitive. Enabling long-lived deployments with proactive defense against key accumulation requires a dynamic resharing protocol that (i) changes guardian sets G_i and/or shares without re-running Generation, and (ii) provides forward secrecy against state compromise. Integrating proactive secret sharing (DPSS) with heterogeneous guardian sets while

preserving public verifiability and succinct proofs is an open direction.

Adaptive and targeted adversaries: So far the analysis assumes static corruption and random churn. Future work might study *adaptive/rushing* behavior and *targeted removals* against the guardian topology $\{G_i\}$. The goal is to quantify how privacy and liveness degrade when the attacker selects corruptions after seeing Round 1 transcripts, and to assess practical mitigations: mild commit–reveal or public randomness to limit key bias, plus topology-aware defenses. Using the same reconstruction predicate R , one could report topology-sensitive lower bounds on the effort needed to break privacy or liveness and identify when targeted strategies matter in practice.

Topology and load control: Hotspots and concentration risks can be reduced by combining local diversity constraints (soft/hard caps per guardian, randomly sampled guardians, overlap-distance limits, reputation that discounts hubs) with a hierarchical FDKG where domains run local FDKG and a second tier aggregates domain keys. Future work might measure how these knobs reshape $\{G_i\}$: lowering $\max_g c_g / \text{median}(c_g)$, bounding worst-case reconstruction traffic, and increasing the minimal reconstruction-capable set for targeted breaks—while preserving heterogeneous trust and near $O(nk)$ per tier.

Broader Applications: Although this paper focuses on an i-voting application, the FDKG concept is a general cryptographic primitive. Its applicability extends to any setting requiring robust, threshold-based secret reconstruction where global membership is uncertain or dynamic. Potential areas include collaborative data encryption/decryption in ad-hoc networks, decentralized credential management systems, and key management for emergent governance structures like Decentralized Autonomous Organizations (DAOs). Exploring these diverse applications and tailoring FDKG to their specific needs presents a rich avenue for future work.

XI. DISCUSSION AND CONCLUSIONS

This paper introduced Federated Distributed Key Generation (FDKG), a novel protocol that enhances the flexibility and resilience of distributed key establishment in dynamic environments. Unlike standard DKG protocols that assume a fixed set of participants and a global threshold, FDKG empowers individual participants to select their own “guardian sets” of size k for share recovery, accommodating scenarios with unknown or fluctuating network membership. Our security analysis (Section V) demonstrates that FDKG achieves correctness and privacy for the key generation phase, and liveness for key reconstruction, provided an adversary does not control both a participant and $k - t + 1$ of its chosen guardians during reconstruction.

FDKG generalizes traditional DKG by integrating principles of federated trust, reminiscent of Federated Byzantine Agreement (FBA) systems like the Stellar Consensus Protocol. This allows for dynamic trust delegation at the participant level. By achieving its core functionality in two communication rounds (generation and reconstruction), FDKG is suitable for

systems involving human interaction, such as the i-voting application presented. The reliance on NIZK proofs (e.g., zk-SNARKs) for verifiability ensures protocol integrity but introduces computational overhead and, in the case of Groth16, a trusted setup requirement, which are important considerations for deployment. Future work could explore NIZKs with transparent setups (e.g., STARKs or Bulletproofs, though the latter might impact succinctness) to alleviate this.

Our simulations (Section VI) provided practical insights into FDKG’s operational robustness. Liveness was shown to be heavily influenced by participant engagement (participation rate p) and continued availability (retention rate r). For example, in a network of $n = 100$ parties with $p = 0.8$ and $r = 0.9$, FDKG consistently achieved liveness for thresholds $t < 0.7k$. However, with a lower retention of $r = 0.5$, perfect liveness was only achieved for smaller thresholds $t \leq 0.4k$ (Figure 5). The choice of guardian selection topology also matters, with Barabási-Albert (BA) providing a slight resilience advantage over random selection in smaller networks due to its hub-centric nature (Figure 6), although this advantage diminishes for $n > 1000$.

Performance evaluations (Section VIII) quantified the computational and communication costs. For $n = 100$ participants with $k = 40$ guardians, the FDKG Generation phase incurred approximately 336 kB of broadcast data per participant. NIZK proof generation, while enabling verifiability, is the main computational bottleneck, with π_{FDKG_i} proving times ranging from 3.36s (for $k = 10$) to 14.79s (for $k = 100$) in our experiments (Table III). These figures highlight the trade-off between the flexibility offered by larger guardian sets and the associated costs, underscoring the need for careful parameter tuning based on specific application constraints: smaller (k, t) values may be preferable for latency-sensitive systems, while larger values enhance resilience in more adversarial settings. A critical aspect of FDKG is balancing guardian set size (k) against security; while smaller sets reduce overhead, they concentrate trust, making individual guardians more critical for liveness if the original participant is unavailable or malicious.

We envision FDKG as a versatile cryptographic primitive for multi-party systems that require robust and verifiable key generation but must operate under conditions of incomplete or dynamic participation. Future research directions include optimizing NIZK proofs for FDKG-specific relations, refining vote decoding mechanisms for FDKG-based voting schemes, and developing more sophisticated, context-aware guardian selection strategies to further enhance FDKG’s practical usability and security. By embracing a decentralized and adaptive ethos, FDKG offers a pathway to building scalable and secure distributed systems for diverse collaborative environments.

REPRODUCIBLE RESEARCH

To ensure the reproducibility of our research, all source code, datasets, and scripts used to generate results are publicly available. The repository is hosted at <https://github.com/stanbar/fdkg>.

ACKNOWLEDGMENTS

We thank Lev Soukhanov, Maya Dotan, Jonathan Katz, Oskar Goldhahn, and Thomas Haines for their insightful comments, enhancing the paper’s structure and cryptographic rigor.

The research was supported partially by the project “Cloud Artificial Intelligence Service Engineering (CAISE) platform to create universal and smart services for various application areas”, No. KPOD.05.10-IW.10-0005/24, as part of the European IPCEI-CIS program, financed by NRRP (National Recovery and Resilience Plan).

REFERENCES

- [1] T. P. Pedersen, “A Threshold Cryptosystem without a Trusted Party,” in *Advances in Cryptology — EUROCRYPT ’91*, D. W. Davies, Ed. Springer, 1991, pp. 522–526.
- [2] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, “Secure Distributed Key Generation for Discrete-Log Based Cryptosystems,” in *Advances in Cryptology — EUROCRYPT ’99*, J. Stern, Ed. Springer Berlin Heidelberg, vol. 1592, pp. 295–310. [Online]. Available: http://link.springer.com/10.1007/3-540-48910-X_21
- [3] V. Shoup and R. Gennaro, “Securing threshold cryptosystems against chosen ciphertext attack,” in *Advances in Cryptology — EUROCRYPT ’98*, K. Nyberg, Ed. Springer, pp. 1–16.
- [4] A. Boldyreva, “Threshold Signatures, Multisignatures and Blind Signatures Based on the Gap-Diffie-Hellman-Group Signature Scheme,” in *Public Key Cryptography — PKC 2003*, Y. G. Desmedt, Ed. Springer, pp. 31–46.
- [5] E. Kokoris-Kogias, E. C. Alp, L. Gasser, P. Jovanovic, E. Syta, and B. Ford. CALYPSO: Private Data Management for Decentralized Ledgers. [Online]. Available: <https://eprint.iacr.org/2018/209>
- [6] C. Cachin, K. Kursawe, and V. Shoup. Random Oracles in Constantinople: Practical Asynchronous Byzantine Agreement using Cryptography. [Online]. Available: <https://eprint.iacr.org/2000/034>
- [7] R. Cramer, I. Damgård, and J. B. Nielsen. Multiparty Computation from Threshold Homomorphic Encryption. [Online]. Available: <https://eprint.iacr.org/2000/055>
- [8] I. Damgård and J. B. Nielsen, “Universally Composable Efficient Multiparty Computation from Threshold Homomorphic Encryption,” in *Advances in Cryptology - CRYPTO 2003*, D. Boneh, Ed. Springer, pp. 247–264.
- [9] Z. Brakerski, O. Friedman, A. Marmor, D. Mutzari, Y. Spiizer, and N. Trieu. Threshold FHE with Efficient Asynchronous Decryption. [Online]. Available: <https://eprint.iacr.org/2025/712>
- [10] K. Choi, A. Manoj, and J. Bonneau. SoK: Distributed Randomness Beacons. [Online]. Available: <https://eprint.iacr.org/2023/728>
- [11] S. Das, B. Pinkas, A. Tomescu, and Z. Xiang. Distributed Randomness using Weighted VUFs. [Online]. Available: <https://eprint.iacr.org/2024/198>

- [12] A. Kavousi, Z. Wang, and P. Jovanovic. SoK: Public Randomness. [Online]. Available: <https://eprint.iacr.org/2023/1121>
- [13] R. Gennaro, S. Goldfeder, and A. Narayanan. Threshold-optimal DSA/ECDSA signatures and an application to Bitcoin wallet security. [Online]. Available: <https://eprint.iacr.org/2016/013>
- [14] “LIT-Protocol/whitepaper,” Lit Protocol. [Online]. Available: <https://github.com/LIT-Protocol/whitepaper>
- [15] B. Adida, “Helios: Web-based Open-Audit Voting,” in *USENIX Security Symposium*, vol. 17, pp. 335–348. [Online]. Available: https://www.usenix.org/legacy/event/sec08/tech/full_papers/adida/adida.pdf
- [16] V. Cortier, P. Gaudry, and S. Glondu, “Belenios: A Simple Private and Verifiable Electronic Voting System,” in *Foundations of Security, Protocols, and Equational Reasoning*, J. D. Guttman, C. E. Landwehr, J. Meseguer, and D. Pavlovic, Eds. Springer International Publishing, vol. 11565, pp. 214–238.
- [17] R. Haenni, R. E. Koenig, P. Locher, and E. Dubuis. CHVote Protocol Specification. [Online]. Available: <https://eprint.iacr.org/2017/325>
- [18] S. Barański, B. Biedermann, and J. Ellul. A Trust-Centric Approach To Quantifying Maturity and Security in Internet Voting Protocols. [Online]. Available: <http://arxiv.org/abs/2412.10611>
- [19] D. Mazieres, “The stellar consensus protocol: A federated model for internet-level consensus,” vol. 32, pp. 1–45.
- [20] R. Bacho and A. Kavousi. SoK: Dlog-based Distributed Key Generation. [Online]. Available: <https://eprint.iacr.org/2025/819>
- [21] P.-A. Fouque and J. Stern, “One Round Threshold Discrete-Log Key Generation without Private Channels,” in *Public Key Cryptography*, K. Kim, Ed. Springer Berlin Heidelberg, vol. 1992, pp. 300–316. [Online]. Available: http://link.springer.com/10.1007/3-540-44586-2_22
- [22] A. Kate, G. M. Zaverucha, and I. Goldberg, “Constant-Size Commitments to Polynomials and Their Applications,” in *Advances in Cryptology - ASIACRYPT 2010*, ser. Lecture Notes in Computer Science, M. Abe, Ed. Springer, pp. 177–194.
- [23] K. Gurkan, P. Jovanovic, M. Maller, S. Meiklejohn, G. Stern, and A. Tomescu. Aggregatable Distributed Key Generation. [Online]. Available: <https://eprint.iacr.org/2021/005>
- [24] J. Groth. Non-interactive distributed key generation and key resharing. [Online]. Available: <https://eprint.iacr.org/2021/339>
- [25] A. Kate, E. V. Mangipudi, P. Mukherjee, H. Saleem, and S. A. K. Thyagarajan. Non-interactive VSS using Class Groups and Application to DKG. [Online]. Available: <https://eprint.iacr.org/2023/451>
- [26] I. Cascudo and B. David. Publicly Verifiable Secret Sharing over Class Groups and Applications to DKG and YOSO. [Online]. Available: <https://eprint.iacr.org/2023/1651>
- [27] H. Feng, T. Mai, and Q. Tang. Scalable and Adaptively Secure Any-Trust Distributed Key Generation and All-hands Checkpointing. [Online]. Available: <https://eprint.iacr.org/2023/1773>
- [28] R. Bacho, C. Lenzen, J. Loss, S. Ochsenschreier, and D. Papachristoudis. GRandLine: Adaptively Secure DKG and Randomness Beacon with (Log-)Quadratic Communication Complexity. [Online]. Available: <https://eprint.iacr.org/2023/1887>
- [29] H. Feng, Z. Lu, and Q. Tang. Dragon: Decentralization at the cost of Representation after Arbitrary Grouping and Its Applications to Sub-cubic DKG and Interactive Consistency. [Online]. Available: <https://eprint.iacr.org/2024/168>
- [30] P. Feldman, “A practical scheme for non-interactive verifiable secret sharing,” in *28th Annual Symposium on Foundations of Computer Science (Sfcs 1987)*, pp. 427–438. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/4568297>
- [31] A. Kate, Y. Huang, and I. Goldberg. Distributed Key Generation in the Wild. [Online]. Available: <https://eprint.iacr.org/2012/377>
- [32] S. Das, T. Yurek, Z. Xiang, A. Miller, L. Kokoris-Kogias, and L. Ren, “Practical Asynchronous Distributed Key Generation,” in *2022 IEEE Symposium on Security and Privacy (SP)*, pp. 2518–2534. [Online]. Available: <https://ieeexplore.ieee.org/document/9833584/?arnumber=9833584&tag=1>
- [33] S. Das, Z. Xiang, L. Kokoris-Kogias, and L. Ren. Practical Asynchronous High-threshold Distributed Key Generation and Distributed Polynomial Sampling. [Online]. Available: <https://eprint.iacr.org/2022/1389>
- [34] H. Zhang, S. Duan, C. Liu, B. Zhao, X. Meng, S. Liu, Y. Yu, F. Zhang, and L. Zhu, “Practical Asynchronous Distributed Key Generation: Improved Efficiency, Weaker Assumption, and Standard Model,” in *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pp. 568–581. [Online]. Available: <https://ieeexplore.ieee.org/document/10202657/?arnumber=10202657>
- [35] M. Stadler, “Publicly Verifiable Secret Sharing,” in *Advances in Cryptology — EUROCRYPT ’96*, U. Maurer, Ed. Springer, pp. 190–199.
- [36] B. Schoenmakers, “A Simple Publicly Verifiable Secret Sharing Scheme and its Application to Electronic Voting,” in *Advances in Cryptology—CRYPTO’99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings*. Springer, pp. 148–164.
- [37] B. Hu, Z. Zhang, H. Chen, Y. Zhou, H. Jiang, and J. Liu. DyCAPS: Asynchronous Dynamic-committee Proactive Secret Sharing. [Online]. Available: <https://eprint.iacr.org/2022/1169>
- [38] J. Katz. Round Optimal Fully Secure Distributed Key Generation. [Online]. Available: <https://eprint.iacr.org/2023/1094>
- [39] S. K. D. Maram, F. Zhang, L. Wang, A. Low, Y. Zhang, A. Juels, and D. Song. CHURP: Dynamic-

- Committee Proactive Secret Sharing. [Online]. Available: <https://eprint.iacr.org/2019/017>
- [40] M. Stadler, “Publicly Verifiable Secret Sharing,” in *Advances in Cryptology — EUROCRYPT ’96*, U. Maurer, Ed. Springer Berlin Heidelberg, vol. 1070, pp. 190–199. [Online]. Available: http://link.springer.com/10.1007/3-540-68339-9_17
- [41] R. Bacho and J. Loss. Adaptively Secure (Aggregatable) PVSS and Application to Distributed Randomness Beacons. [Online]. Available: <https://eprint.iacr.org/2023/1348>
- [42] J. Groth, “On the Size of Pairing-Based Non-interactive Arguments,” in *Advances in Cryptology – EUROCRYPT 2016*, M. Fischlin and J.-S. Coron, Eds. Springer, pp. 305–326.
- [43] A. Gabizon, Z. J. Williamson, and O. Ciobotaru. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. [Online]. Available: <https://eprint.iacr.org/2019/953>
- [44] A. Chiesa, Y. Hu, M. Maller, P. Mishra, P. Vesely, and N. Ward. Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS. [Online]. Available: <https://eprint.iacr.org/2019/1047>
- [45] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. [Online]. Available: <https://eprint.iacr.org/2018/046>
- [46] Wei Jie Koh. ElGamal encryption, decryption, and rerandomization, with circom support - zk-s[n]tarks. Ethereum Research. [Online]. Available: <https://ethresear.ch/t/elgamal-encryption-decryption-and-rerandomization-with-circom-support/8074>
- [47] K. W. Jie, “Weijiekoh/elgamal-babyjub.” [Online]. Available: <https://github.com/weijiekoh/elgamal-babyjub>
- [48] S. Steffen, B. Bichsel, R. Baumgartner, and M. Vechev, “Zeestar: Private smart contracts by homomorphic encryption and zero-knowledge proofs,” in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 179–197. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9833732/>
- [49] R. Canetti, “Universally composable security: A new paradigm for cryptographic protocols,” in *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, pp. 136–145. [Online]. Available: <https://ieeexplore.ieee.org/document/959888>
- [50] J. Katz and Y. Lindell, *Introduction to Modern Cryptography: Third Edition*, 3rd ed. Chapman and Hall/CRC.
- [51] B. Schoenmakers, “Lecture notes cryptographic protocols,” vol. 1.
- [52] F. Hao, P. Y. Ryan, and P. Zieliński, “Anonymous voting by two-round public discussion,” vol. 4, no. 2, pp. 62–67. [Online]. Available: http://homepages.cs.ncl.ac.uk/feng.hao/files/OpenVote_IET.pdf
- [53] O. Baudron, P.-A. Fouque, D. Pointcheval, J. Stern, and G. Poupard, “Practical multi-candidate election system,” in *Proceedings of the Twentieth Annual ACM Symposium on Principles of Distributed Computing*, pp. 274–283.
- [54] ERC-4337: Account Abstraction Using Alt Mempool. Ethereum Improvement Proposals. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-4337>

APPENDIX

A. Ideal Functionality $\mathcal{F}_{FDKG}$

The ideal functionality $\mathcal{F}_{FDKG}$ interacts with the parties $P_1, \dots, P_n$ and a simulator $\mathcal{S}$. It is parameterized by system parameters (group $\mathbb{G}$, generator G), FDKG parameters (t, k) , and dynamically tracks participation. Its formal description is provided in Figure 9.

B. Proof Sketch for Theorem 1

We show that Π_{FDKG} securely realizes $\mathcal{F}_{FDKG}$ against any PPT adversary $\mathcal{A}$ corrupting a static set $\mathcal{C}$. The proof proceeds by a sequence of hybrids changing the honest parties’ Round 1 messages.

a) *Notation:* Write View_r for $\mathcal{A}$ ’s view in world r . All probabilities are over the joint coin tosses of honest parties, the adversary, and the NIZK/PKE.

Hybrid H_0 (Real world). Honest parties follow Π_{FDKG} : they publish $(E_i, \mathcal{G}_i, \{C_{i,j}\}_{j \in \mathcal{G}_i}, \pi_{FDKG_i})$ where each $C_{i,j}$ encrypts the *real* share $f_i(j)$ and π_{FDKG_i} is a real proof.

Hybrid H_1 (Simulated NIZKs). Same as H_0 , except every honest π_{FDKG_i} is replaced by a simulated proof produced by the NIZK simulator for the same public statement.

Lemma 1 (Zero-knowledge step). $\text{View}_{H_0} \approx_c \text{View}_{H_1}$ under NIZK zero-knowledge.

Hybrid H_2 (IND-CPA switch for honest recipients). Same as H_1 , except that for every honest dealer i and honest recipient $j \notin \mathcal{C}$, the ciphertext $C_{i,j}$ is changed from an encryption of $f_i(j)$ to an encryption of 0 under pk_j . For corrupted recipients $j \in \mathcal{C}$, honest dealers still encrypt the *actual* share $f_i(j)$.

Lemma 2 (IND-CPA step). $\text{View}_{H_1} \approx_c \text{View}_{H_2}$ under PKE IND-CPA.

Hybrid H_3 (Ideal world distribution). This is the ideal execution with simulator $\mathcal{S}$ interacting with $\mathcal{F}_{FDKG}$:

- For corrupt dealers, the simulator relays their real broadcasts.
- For each honest dealer i , the simulator uses the public E_i and $\mathcal{G}_i$ supplied by $\mathcal{F}_{FDKG}$, encrypts 0 for honest recipients, and encrypts the *actual* shares $f_i(j)$ (obtained from $\mathcal{F}_{FDKG}$) for corrupted recipients $j \in \mathcal{C}$. Proofs are simulated.

By construction, View_{H_2} and View_{H_3} are *identical*.

Lemma 3 (Identity step). $\text{View}_{H_2} \equiv \text{View}_{H_3}$.

b) *Indistinguishability conclusion:* By Lemmas 1–3 and a triangle inequality, $\text{View}_{H_0} \approx_c \text{View}_{H_3}$, so the real execution is computationally indistinguishable from the ideal one.

Ideal Functionality $\mathcal{F}_{FDKG}^{n,t,k}$

1. Initialization:

- Waits to receive (Init, $\mathcal{C}$) from the simulator $\mathcal{S}$. Stores the set of corrupted parties $\mathcal{C}$. Let $\mathcal{H} = \mathbb{P} \setminus \mathcal{C}$.
- Initializes an empty set of participants $\mathbb{D} = \emptyset$.
- Initializes empty storage for polynomials f_i and guardian sets $\mathcal{G}_i$ for all $i \in \mathbb{P}$.
- Initializes state flag 'phase' = 'activation'.

2. Honest Party Activation: Upon receiving (ActivateHonest, $i, \mathcal{G}_i$) from the simulator $\mathcal{S}$ (indicating honest party $P_i \in \mathcal{H}$ decides to participate):

- If 'phase' is 'activation' and $P_i \notin \mathbb{D}$:
 - Add P_i to $\mathbb{D}$.
 - Store the provided guardian set $\mathcal{G}_i$ (verify $|\mathcal{G}_i| = k$, $P_i \notin \mathcal{G}_i$).
 - Choose a random polynomial $f_i(X)$ of degree $t-1$ over $\mathbb{Z}_q$ and store it.
 - Send (Activated, i) back to the simulator $\mathcal{S}$.

3. Corrupt Party Activation: Upon receiving (ActivateCorrupt, $i, f_i, \mathcal{G}_i$) from the simulator $\mathcal{S}$ (on behalf of $P_i \in \mathcal{C}$):

- If 'phase' is 'activation' and $P_i \notin \mathbb{D}$:
 - Add P_i to $\mathbb{D}$.
 - Verify degree of f_i is $t-1$, $|\mathcal{G}_i| = k$, $P_i \notin \mathcal{G}_i$. Store valid f_i and $\mathcal{G}_i$.

4. Key Computation Trigger: Upon receiving (ComputeKeys) from the simulator $\mathcal{S}^2$:

- If 'phase' is 'activation':
 - Set 'phase' = 'computed'.
 - If $\mathbb{D} = \emptyset$: Send (KeysComputed, null, $\emptyset, \emptyset, \emptyset, \emptyset$) to $\mathcal{S}$ and stop this trigger processing.
 - For each $P_k \in \mathbb{D}$: Define $d_k := f_k(0)$ and $E_k := G^{d_k}$. Let $\mathbb{E} = \{E_k\}_{k \in \mathbb{D}}$.
 - Compute global secret key $\mathbf{d} := \sum_{k \in \mathbb{D}} d_k \pmod{q}$ and global public key $\mathbf{E} := G^{\mathbf{d}}$.
 - For each $k \in \mathbb{D}$ and each j s.t. $P_j \in \mathcal{G}_k$: Compute share $[d_k]_j := f_k(j)$.
 - Send (KeysComputed, $\mathbf{E}, \mathbb{E}, \{\mathcal{G}_k\}_{k \in \mathbb{D} \cap \mathcal{H}}, \{(d_k, f_k, \mathcal{G}_k)\}_{k \in \mathbb{D} \cap \mathcal{C}}, \{[d_k]_j \mid k \in \mathbb{D}, P_j \in \mathcal{G}_k \cap \mathcal{C}\}$) to the simulator $\mathcal{S}$.
 - For each honest participant $P_i \in \mathbb{D} \cap \mathcal{H}$: Send (Output, $\mathbf{E}, \mathbb{E}, d_i, \mathcal{G}_i, \{[d_k]_i \mid k \in \mathbb{D}, P_i \in \mathcal{G}_k\}$) to P_i .
-

Figure 9: Ideal functionality for Federated Distributed Key Generation $\mathcal{F}_{FDKG}^{n,t,k}$.

c) Correctness and Robustness from soundness: In any world where honest broadcasts are *accepted*, NIZK *soundness* implies that each accepted tuple of an (honest or corrupt) dealer i is consistent with a single degree- $(t-1)$ polynomial f_i and with $E_i = G^{f_i(0)}$; otherwise the proof would be rejected except with negligible probability. Hence each accepted ciphertext encrypts $f_i(j)$, and the published E_i equals G^{d_i} with $d_i = f_i(0)$, yielding $\mathbf{E} = \prod_{i \in \mathbb{D}} E_i = G^{\sum_{i \in \mathbb{D}} d_i}$. Moreover, verification rejects malformed or equivocal broadcasts, so corrupted parties cannot force acceptance of inconsistent data; their only effect is to omit or abort. This establishes *Correctness* and *Robustness* of Generation as stated in Theorem 1.

d) Privacy of Generation: In H_2 (hence in H_3), the adversary's view on honest-to-honest ciphertexts is distributed as encryptions of 0; by Lemma 2, this is indistinguishable from encryptions of the true shares. Ciphertexts to corrupted recipients contain the *real* shares, matching the ideal leakage $\{[d_i]_j \mid j \in \mathcal{G}_i \cap \mathcal{C}\}$. Simulated proofs are indistinguishable by Lemma 1. Therefore the view leaks no additional information about honest $\{d_i\}$ beyond public values and the intended shares to corrupted recipients, establishing *Privacy* for Generation.

This section details the specific statements and witnesses for the NIZK proofs used in the FDKG protocol and the associated voting application. We assume a NIZK proof sys-

tem (Setup, Prove, Verify) operating with a common reference string crs over a suitable group $\mathbb{G}$ with generator G and order q . The proofs are instantiated using Groth16 [42]. Each subsection defines the relation $\mathcal{R}$ implicitly through the statement x and witness w . Proofs are generated as $\pi \leftarrow \text{Prove}(\text{crs}, x, w)$ and verified by checking $\text{Verify}(\text{crs}, x, \pi) \stackrel{?}{=} 1$.

The detailed Circom circuit implementations for these proofs are publicly available on <https://github.com/stanbar/FDKG>.

C. FDKG Proof (π_{FDKG_i})

Proves correct generation and encryption of shares by participant P_i during the Generation phase.

- **Statement (x_i):** Consists of the party's partial public key E_i , the set of PKE public keys $\{\text{pk}_j\}_{P_j \in \mathcal{G}_i}$ for its guardians, and the set of corresponding encrypted shares $\mathbb{C}_i = \{C_{i,j}\}_{P_j \in \mathcal{G}_i}$.
- **Witness (w_i):** Consists of the coefficients $\{a_0, \dots, a_{t-1}\}$ of P_i 's secret polynomial $f_i(X)$ (where $d_i = a_0 = f_i(0)$), and the PKE randomness $\{(k_{i,j}, r_{i,j})\}_{P_j \in \mathcal{G}_i}$ used to produce each ciphertext in $\mathbb{C}_i$.
- **Relation $\mathcal{R}_{FDKG}((x_i), w_i) \in \mathcal{R}_{FDKG}$:** The circuit (PVSS.circom) checks that:

- 1) The public E_i is correctly derived from the witness coefficient a_0 (i.e., $E_i = G^{a_0}$).

2) For each guardian $P_j \in \mathcal{G}_i$:

- The plaintext share $s_{i,j} = f_i(j)$ is correctly evaluated from the witness polynomial coefficients $\{a_k\}$.
- The ciphertext $C_{i,j} \in \mathbb{C}_i$ is a valid ElGamal variant encryption of the plaintext share $s_{i,j}$ under guardian P_j 's PKE public key pk_j , using the witness randomness $(k_{i,j}, r_{i,j})$. This involves verifying the structure $C_1 = G^{k_{i,j}}$, $M = G^{r_{i,j}}$, $C_2 = (pk_j)^{k_{i,j}} \cdot M$, and $\Delta = M \cdot x - s_{i,j}$.

- **Circuit Implementation:** <https://github.com/stanbar/FDKG/blob/main/circuits/circuits/pvss.circom>

D. Partial Secret Proof (π_{PS_i})

Proves knowledge of the partial secret key d_i corresponding to the partial public key E_i during reconstruction.

- **Statement** (x_i): The partial public key E_i .
- **Witness** (w_i): The partial secret key d_i .
- **Relation** $\mathcal{R}_{PS}$ ($((x_i), w_i) \in \mathcal{R}_{PS}$): Checks that $E_i = G^{d_i}$.
- **Circuit Implementation:** https://github.com/stanbar/FDKG/blob/main/circuits/circuits/elGamal_c1.circom

E. Share Decryption Proof ($\pi_{SPS_{j,i}}$)

Proves correct decryption of an encrypted share $C_{j,i}$ by the recipient party P_i (guardian) using its PKE secret key sk_i .

- **Statement** ($x_{j,i}$): Consists of the encrypted share $C_{j,i} = (C_1, C_2, \Delta)$, the recipient's (guardian P_i 's) PKE public key pk_i , and the claimed plaintext share $s_{j,i}$.
- **Witness** ($w_{j,i}$): The recipient's (guardian P_i 's) PKE secret key sk_i .
- **Relation** $\mathcal{R}_{SPS}$ ($((x_{j,i}), w_{j,i}) \in \mathcal{R}_{SPS}$): The circuit (decrypt_share.circom) checks that applying the ElGamal variant decryption steps using sk_i to (C_1, C_2, Δ) yields the plaintext $s_{j,i}$. That is, it computes $M = C_2 \cdot (C_1^{sk_i})^{-1}$ and verifies $M \cdot x - \Delta = s_{j,i} \pmod{q}$.
- **Circuit Implementation:** https://github.com/stanbar/FDKG/blob/main/circuits/circuits/decrypt_share.circom

F. Ballot Proof (π_{Ballot_i})

Proves a ballot B_i is a valid ElGamal encryption of an allowed vote v_i under the global public key $\mathbf{E}$.

- **Statement** (x_{B_i}): The global public key $\mathbf{E}$ and the ballot $B_i = (C1_i, C2_i)$.
- **Witness** (w_{B_i}): The blinding factor $r_i \in \mathbb{Z}_q$ and the encoded vote message M_{vote} .
- **Relation** $\mathcal{R}_{Ballot}$ ($((x_{B_i}), w_{B_i}) \in \mathcal{R}_{Ballot}$): The circuit (encrypt_ballot.circom) checks that:
 - 1) $C1_i = G^{r_i}$.
 - 2) $C2_i = \mathbf{E}^{r_i} \cdot G^{M_{vote}}$.
 - 3) The witness M_{vote} corresponds to one of the allowed vote encodings (e.g., $M_{vote} \in \{2^0, \dots, 2^{(c-1)m}\}$).
- **Circuit Implementation:** https://github.com/stanbar/FDKG/blob/main/circuits/circuits/encrypt_ballot.circom

G. Partial Decryption Proof (π_{PD_i})

Proves correct computation of the partial decryption PD_i by party P_i .

- **Statement** (x_{PD_i}): The aggregated ElGamal component $C1$, the computed partial decryption PD_i , and the party's partial public key E_i .
- **Witness** (w_{PD_i}): The party's partial secret key d_i .
- **Relation** $\mathcal{R}_{PD}$ ($((x_{PD_i}), w_{PD_i}) \in \mathcal{R}_{PD}$): The circuit (partial_decryption.circom) checks that:
 - 1) $E_i = G^{d_i}$.
 - 2) $PD_i = (C1)^{d_i}$.
- **Circuit Implementation:** https://github.com/stanbar/FDKG/blob/main/circuits/circuits/partial_decryption.circom

H. Partial Decryption Share Proof ($\pi_{PDS_{j,i}}$)

Proves correct decryption of an SSS share $s_{j,i}$ (originally from P_j , decrypted by guardian P_i) and its application to computing P_i 's share of the partial decryption of $C1$.

- **Statement** ($x_{PDS_{j,i}}$): The aggregated ElGamal component $C1$, the computed share of partial decryption $[PD_j]_i$, the encrypted SSS share $C_{j,i} = (C_1, C_2, \Delta)$, and the decrypting guardian P_i 's PKE public key pk_i .
- **Witness** ($w_{PDS_{j,i}}$): The decrypting guardian P_i 's PKE secret key sk_i .
- **Relation** $\mathcal{R}_{PDS}$ ($((x_{PDS_{j,i}}), w_{PDS_{j,i}}) \in \mathcal{R}_{PDS}$): The circuit (partial_decryption_share.circom) first internally computes the plaintext SSS share $s_{j,i} = \text{Dec}_{sk_i}(C_{j,i})$ (verifying the decryption step) and then checks that $[PD_j]_i = (C1)^{s_{j,i}}$.
- **Circuit Implementation:** https://github.com/stanbar/FDKG/blob/main/circuits/circuits/partial_decryption_share.circom